Q. What is a collection?

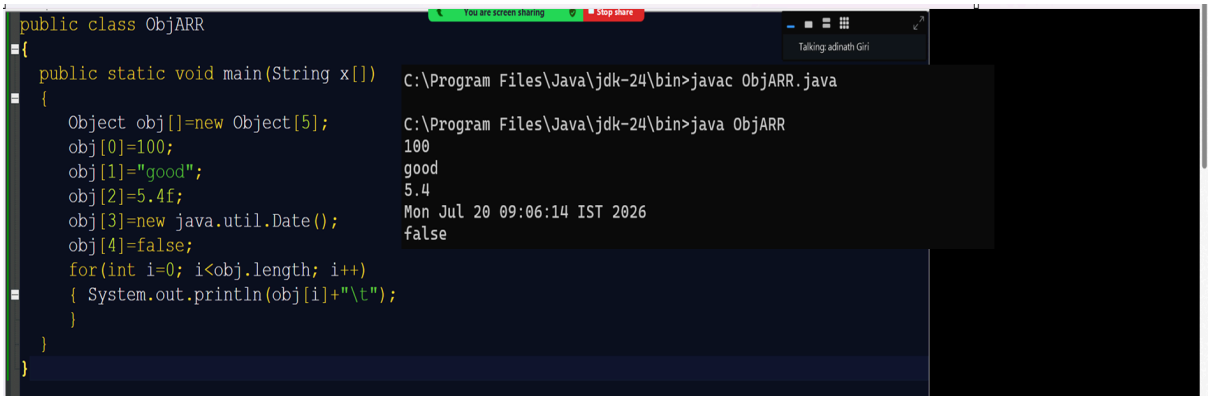
Collection is an inbuilt ready made API provided by JAVA as data structure implementation called as Collection.

Q. Why use Collection or What are the benefits of Collection Framework?

---

1. Ability to store any kind of data or any type of data
2. Dynamic size means collection can increase its size at runtime as well as shrink its size at runtime
3. Collection provide data structure implementation means developer not need to implement data structure manually

Important point: In java if we create an array of object class then we can store different types of data in it.

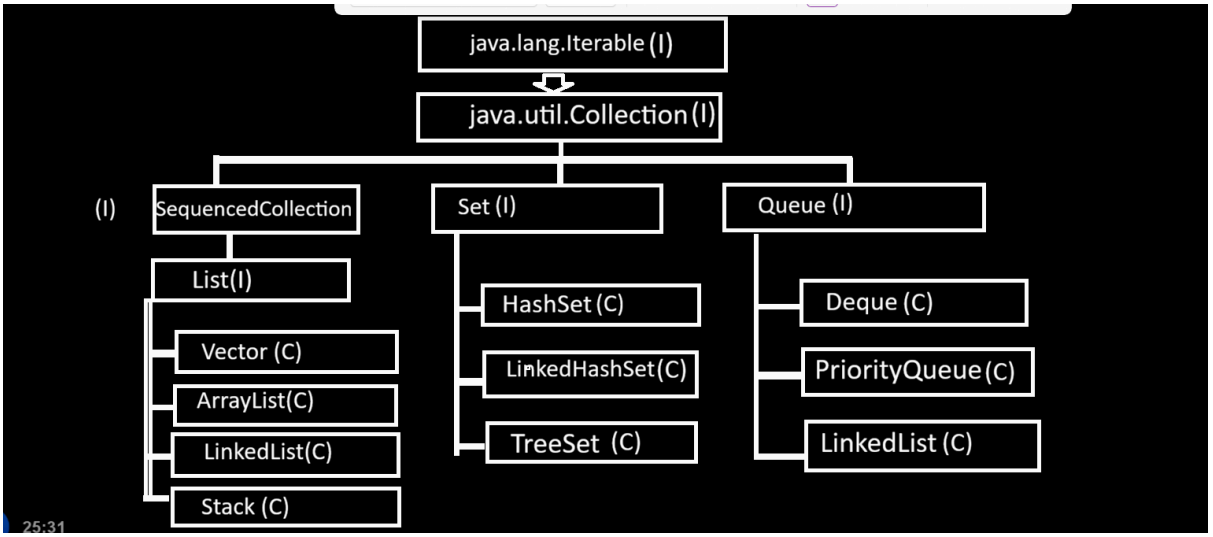


If we think about above code there is some limitations

a. Need to perform data structure logical manually
b. We cannot change size of array at runtime or dynamically

For resolve this problem java suggest use collection framework

If we want to work with collection framework we have inbuilt classes and interface hierarchy



Note: if we think about above collection Hierarchy we have top most interface in hierarchy known as java.lang.Iterable

Q. What is the purpose of the Iterable interface?

The purpose of Iterable interface is fetch data from collection and data fetch or data retrieval operation is common in all collection framework

Structure of Iterable interface

```
public interface java.lang.Iterable<T> {
  public abstract java.util.Iterator<T> iterator();
  public default void forEach(java.util.function.Consumer<? super T>);
  public default java.util.Spliterator<T> spliterator();
}
```

If we think about Iterable interface we have three methods in it

Note: forEach() and splitIterator() these methods we will discuss in JDK 1.8 version topic in JAVA.

Now we want to discuss about iterator method

Iterator iterator(): this method is used for fetch data from collection framework and return reference of Iterator interface

Q. What is the Iterator interface?

Iterator interface is used for fetch data from collection in forward direction and it is known as universal cursor

Iterator interface structure

```
public interface java.util.Iterator<E> {
  public abstract boolean hasNext();
  public abstract E next();
  public default void remove();
  public default void forEachRemaining(java.util.function.Consumer<? super E>);
}
```

Method description of Iterator interface

---

public abstract boolean hasNext(): this method is used for check collection containing data or not if collection contains data return true otherwise return false.

public abstract E next(): this method is used for fetch data from collection and move cursor next element or pointer on next element

public default void remove(): this method is used for remove data from collection

Collection interface

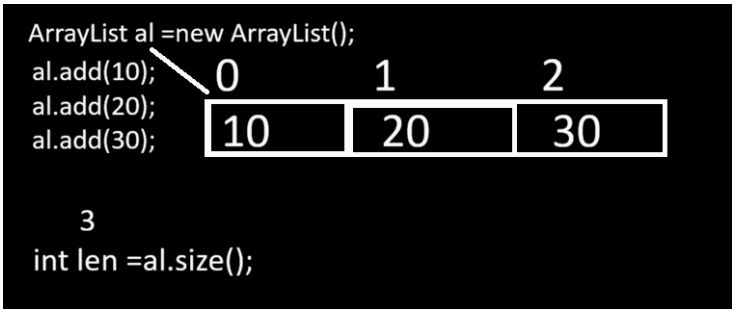
Collection interface is a child of Iterable and parent of all other collection or implementer classes.

Collection interface contain some common methods required for collection framework or common operation required to all collection classes

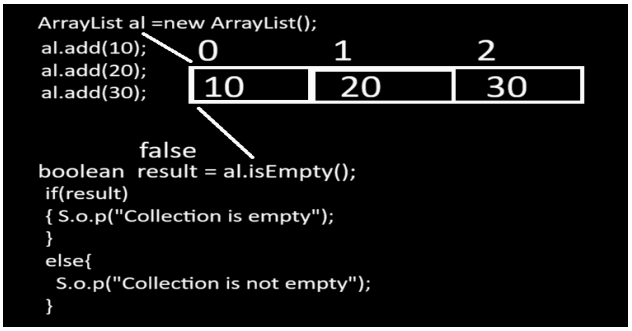
Methods of Collection interface

---

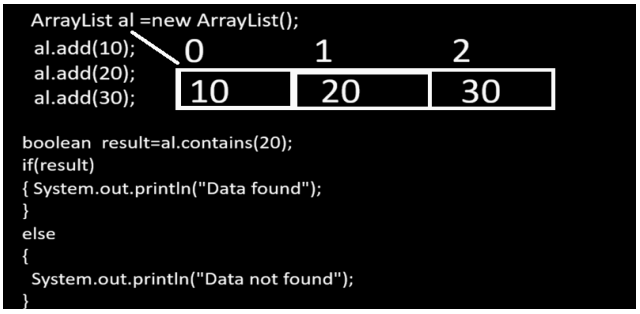
public abstract int size(): this method return number of element present in collection framework



public abstract boolean isEmpty(): this method checks if the collection is empty or not if empty return true otherwise return false.



public abstract boolean contains(java.lang.Object): this method checks if an element is present in collection or not if present return true otherwise return false.
This method is used for search data in collection

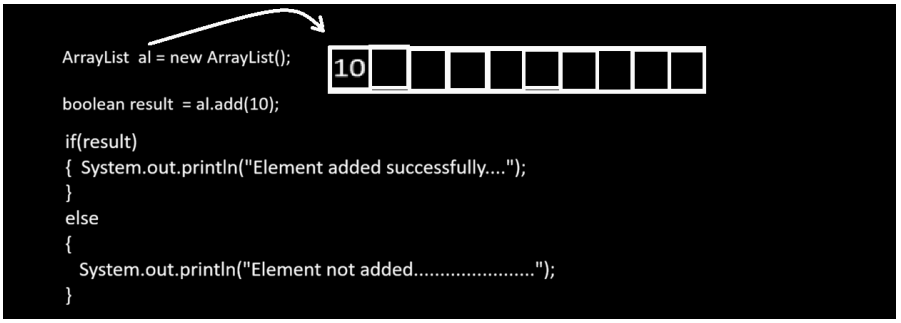


public abstract java.util.Iterator<E> iterator(): this method is used for fetching data from the collection framework and this is originally method of Iterable interface and Iterable is parent of Collection interface

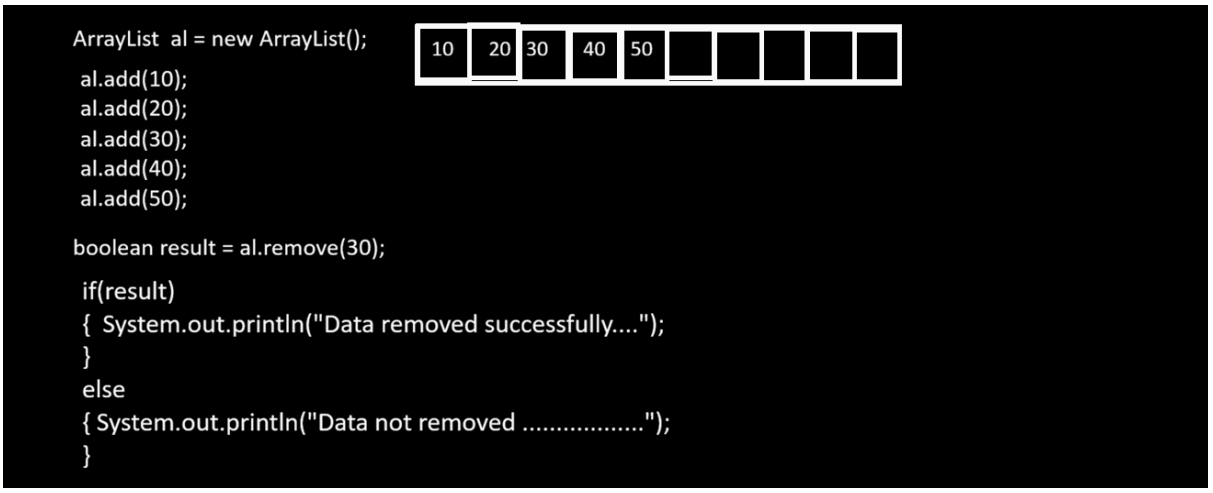
public abstract java.lang.Object[] toArray(): this method is used for converting your collection object to the object class array.



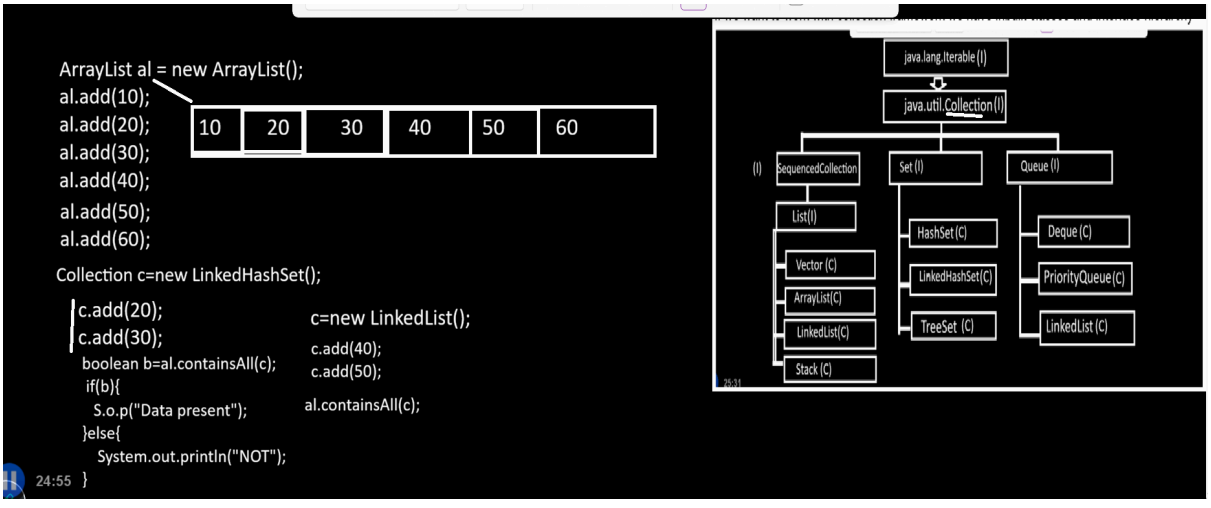
public abstract boolean add(E): this method is used for adding new elements in a collection and if elements added successfully return true otherwise return false.



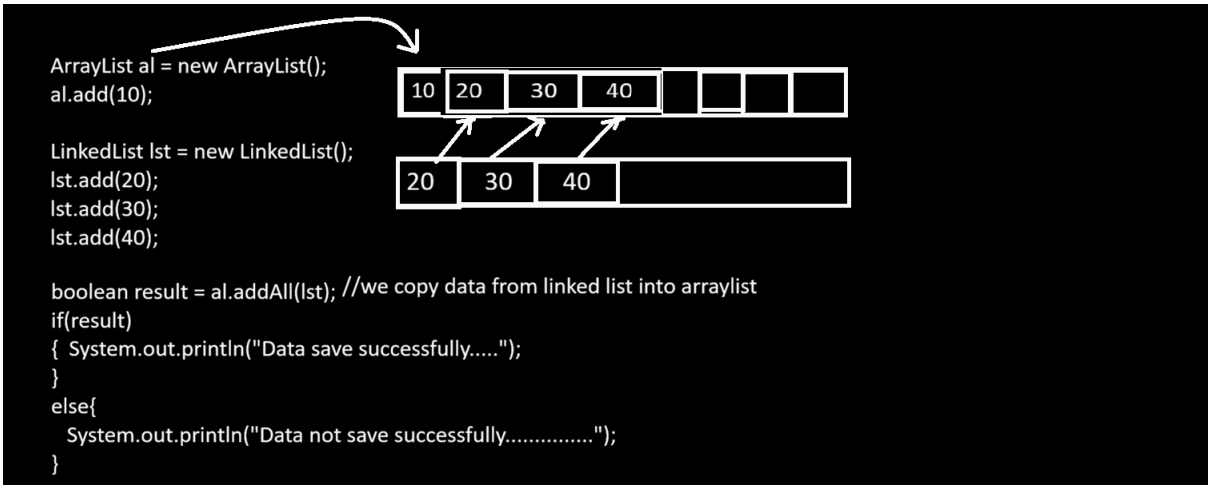
public abstract boolean remove(java.lang.Object): this method is used for removing elements from collection and returning true if element removed successfully otherwise return false.



public abstract boolean containsAll(java.util.Collection<?>): this method is used for searching more than one element in a collection at a time if any one element found in collection returns true otherwise return false.



public abstract boolean addAll(java.util.Collection<? extends E>): this method is used for copying data from another collection or this method helps us add more than one data at time.



public abstract boolean removeAll(java.util.Collection<?>): this method is used for remove more than one elements from collection and return true if elements removed successfully otherwise not

public abstract void clear(): this method can remove all data from collection

Note: remaining these methods we will discuss after object class and after JDK 1.8 version feature

public abstract boolean equals(java.lang.Object);
public abstract int hashCode();
public default java.util.Spliterator<E> spliterator();
public default java.util.stream.Stream<E> stream();
public default java.util.stream.Stream<E> parallelStream();

Note: After collection interface we have three types of collections

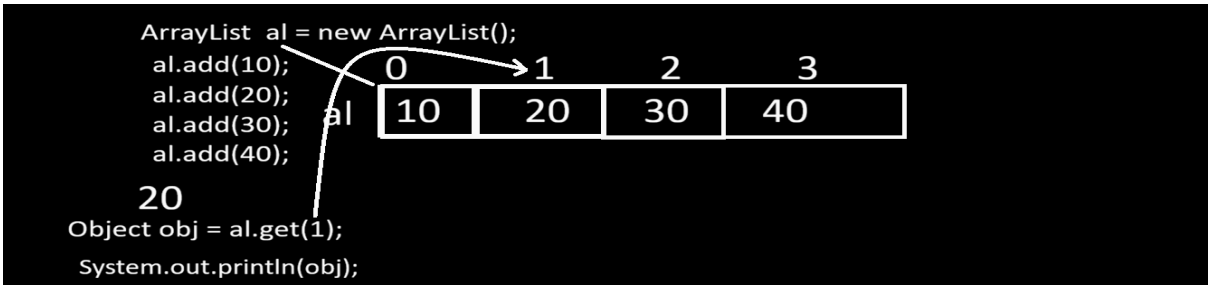
1. **List** : list collection is used for storing unlimited data or infinite data as well as allowing duplicated data and managing data by using indexing techniques.
2. **Set :** Set collection is used for store unique data but arrange data using hashing technique as well as can generate random data
3. **Queue :** Queue collection allowing duplicated data but arrange data in first in first out manner or use the min heap concept for arrange data if we think priority queue

Now we want to discuss about List collection

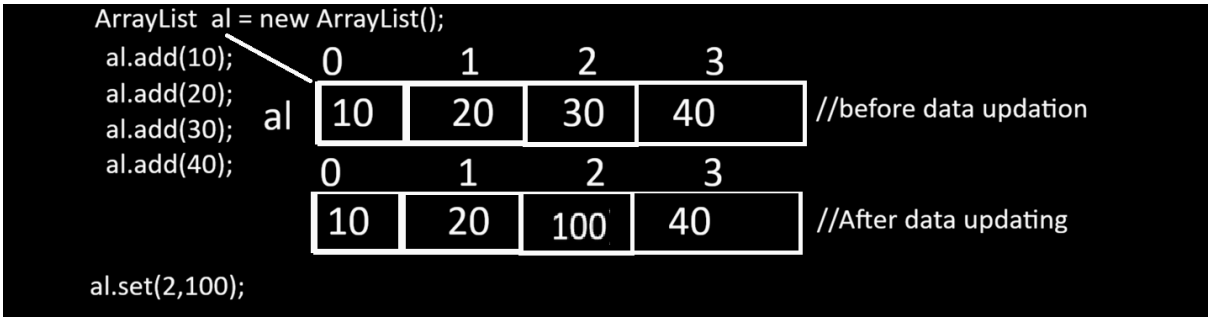
Methods of List collection

---

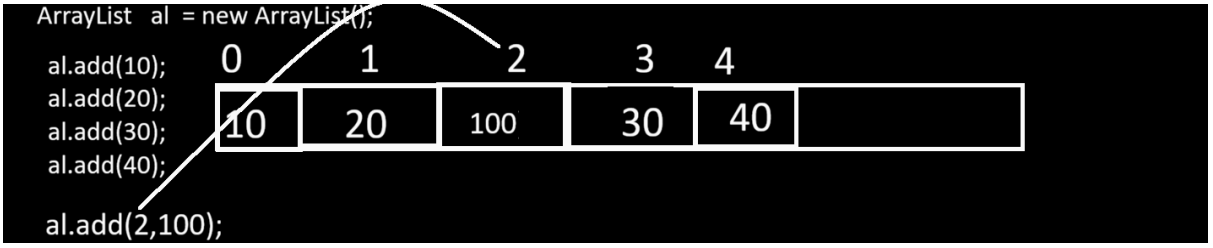
public abstract E get(int): This method is used for returning data from collection by using its index and if index not found return IndexOutOfBoundsException



public abstract E set(int, E): this method is used to replace data in collection by using a specified index.

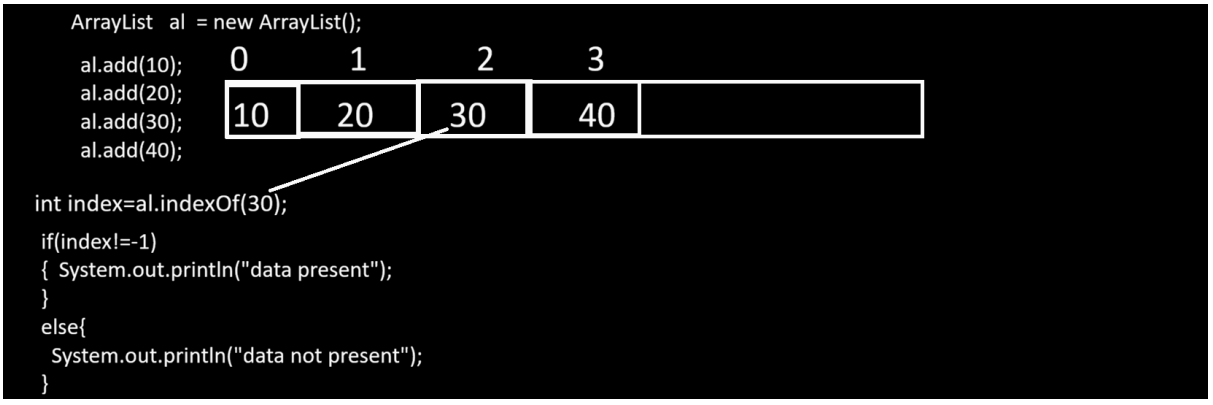


public abstract void add(int, E): This method is used for adding new data in collection or specially in list collection by using a specified index.

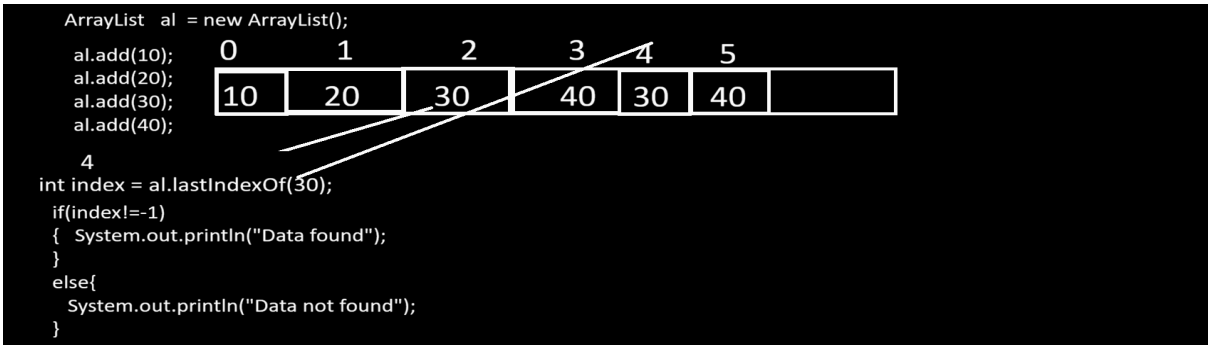


public abstract E remove(int): this method can remove data from collection using an index and if the index is not found generate IndexOutOfBoundsException at runtime.

public abstract int indexOf(java.lang.Object): this method is used for searching purpose in collection and if data found return index of data otherwise returns -1
This method return first occurence of data in collection

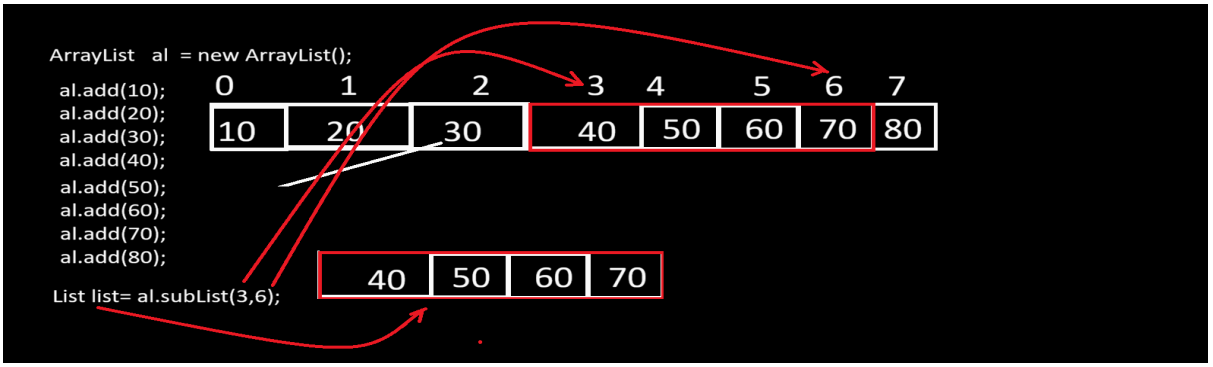


public abstract int lastIndexOf(java.lang.Object): this method returns last occurrence of data from collection if data found multiple times and return index of data if data present in collection framework otherwise return -1.



public abstract java.util.ListIterator<E> listIterator(int): this method return reference of ListIterator and ListIterator can fetch data in forward direction as well as in backward direction but this interface can only works with list collection

public abstract java.util.List<E> subList(int, int): this method is used for extracting data between specified indexes of a list and creating a new list.



public default void addFirst(E): add data at beginning in list collection
public default void addLast(E): add data at last in list collection
public default E getFirst():retrieve first data list from collection
public default E getLast(): retrieve last data from list collection
public default E removeFirst();remove first data from list collection
public default E removeLast();: remove last data from list collection

Vector class

Vector is a dynamic array and it is known as legacy collection or legacy class in Java. Dynamic array means vector can grow size at runtime as well as shrink size at runtime.

Q. What is legacy collection or legacy classes in JAVA?

Legacy collection means those collection classes present in java before collection but later they added as part of collection called as legacy collection

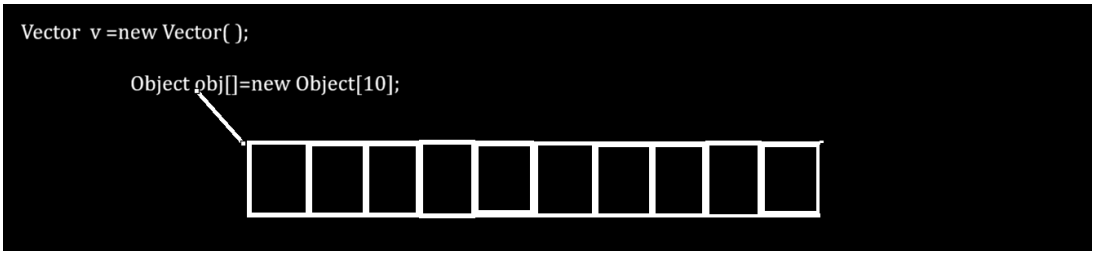
Example: if we think about Vector it is present in java from JDK 1.0 version and Collection Framework launch in Java JDK1.2

Note: if we want to work with Vector we have some inbuilt constructor provided by Vector to us

Constructor of Vector class

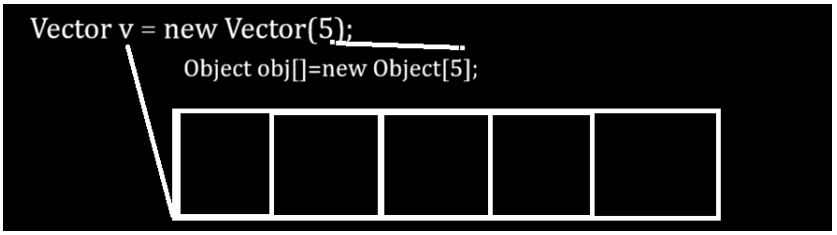
---

a. **Vector():** This constructor creates a Vector with initial capacity 10 means internally when we use this constructor Java creates an object class array with capacity 10.
So we can say the default capacity of Vector is 10.



Note: When Vector cross its capacity then Vector allocate memory than its current capacity

b. **Vector(int initialCapacity):** this constructor is used to create Vector with initial capacity according to use choice.

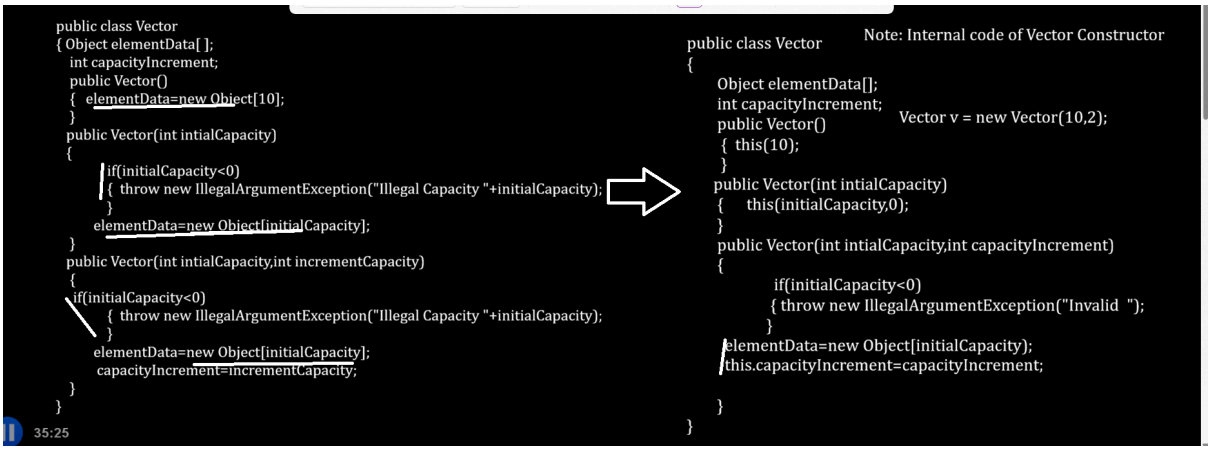


c. **Vector(int initialCapacity,int incrementalCapacity):** This constructor help us provide initial capacity as per user choice as well as provide incremental capacity

int initialCapacity: this parameter help us to set initial capacity of Vector
int incrementalCapacity: this parameter help us to set incremental capacity of Vector

d. **Vector(Collection ref)**: this constructor helps us copy the data from another collection and store it in the Vector object.

Note: Internally java use the constructor chaining concept to create Object class array with Vector constructors



Example with source code

```
package org.techhub;
import java.util.*;
public class VectorApplication {

	public static void main(String[] args) {
		Vector v=new Vector();
		System.out.println("Initial capacity of Vector "+v.capacity());
		v.add(10);
		v.add(20);
```

```
                v.add(30);
                v.add(40);
                v.add(10);
                v.add(20);
                v.add(30);
                v.add(40);
                v.add(10);
                v.add(20);
                v.add(30);
                v.add(40);
                System.out.println("size of vector is  "+v.size());
                System.out.println("After capacity increment "+v.capacity());
        }
}
```

Example of Vector with : Vector(init initialCapacity) constructor

---

```
package org.techhub;
import java.util.*;
public class VectorApplication {

        public static void main(String[] args) {
                Vector v=new Vector(4);
                System.out.println("Initial capacity of Vector "+v.capacity());
                v.add(10);
                v.add(20);
                v.add(30);
                v.add(40);
                v.add(10);
                v.add(20);
                v.add(30);
                v.add(40);
                v.add(10);
                v.add(20);
                v.add(30);
                v.add(40);
                System.out.println("size of vector is  "+v.size());
                System.out.println("After capacity increment "+v.capacity());
        }
}
```

Example of Vector using Vector(int initialCapacity,int capacityIncrement)

```
package org.techhub;
import java.util.*;
public class VectorApplication {
        public static void main(String[] args) {
                Vector v=new Vector(4,2);
```

```
                System.out.println("Initial capacity of Vector "+v.capacity());
                v.add(10);
                v.add(20);
                v.add(30);
                v.add(40);
                v.add(10);
                v.add(20);
                v.add(30);
                v.add(40);
                v.add(10);
                v.add(20);
                v.add(30);
                v.add(40);
                v.add(50);
                System.out.println("size of vector is  "+v.size());
                System.out.println("After capacity increment "+v.capacity());
        }
}
```

Example using Vector(Collection) parameter

```
package org.techhub;
import java.util.*;
public class VectorApplication {
        public static void main(String[] args) {
                ArrayList al= new ArrayList();
                al.add(40);
                al.add(20);
                al.add(30);
                al.add(40);
                al.add(50);
                Vector v=new Vector(al);
                System.out.println(v);
        }
}
```

Important points related with Vector

---

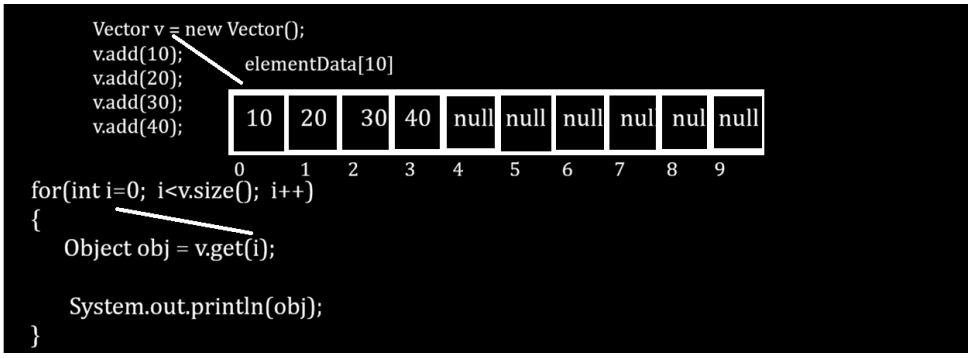
1. Vector is a legacy collection
2. Vector is a dynamic array
3. Vector is an implementer class of List
4. Vector is thread safe i.e asynchronous
5. Vector occupy double memory than its current capacity when capacity cross
6. Default capacity of Vector is 10

Note: if we think about the collection framework when we add data in collection then data is added as Object and when we retrieve data from collection we get data in the form Object class.

How to Fetch data from collection /cursor in collection

---

1. **Using loop :** if we use the loop then using index we can fetch data from collection but it is applicable with only list collection or indexed collection




```
package org.techhub;
import java.util.*;
public class VectorApplication {
	public static void main(String[] args) {
		Vector v=new Vector();
		v.add(10);
		v.add(20);
		v.add(30);
		v.add(40);
		for(int i=0; i<v.size();i++) {
			Object obj = v.get(i);
			System.out.println(obj);
		}
	}
}
```

2. **Using Enumeration :** Enumeration is an interface and which is used to fetch data from legacy collection only and it is read only cursor

How to use Enumeration

Steps.

a. **Create reference of Enumeration interface**

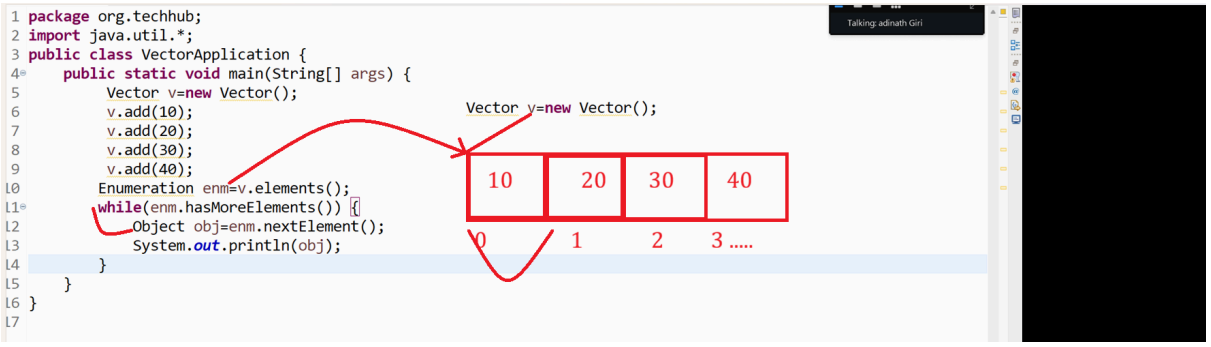
If we want to create reference of Enumeration interface we have elements() method present in Legacy classes

Syntax : Enumeration ref = collref.elements();

b. **Use its method to fetch data from collection**

Methods of Enumeration

a. **boolean hasMoreElements():** this method is used for checking data present in collection or not if present return true otherwise return false.
b. **Object nextElement():** this method is used for fetching data from collection and move cursor on next element



3. **Using Iterator :** Iterator can fetch data from legacy as well as non legacy collection and iterator can remove data from collection at the time data fetching
   Iterator known as universal cursor means work with indexed collection as well as non index collection means work with any collection

   Steps to use iterator

   a. **Create reference of Iterator**
      Syntax: Iterator ref= collref.iterator();
   b. **Use Methods of Iterator interface**
      boolean hasNext(): this method is used for check data present in collection or not if present return true otherwise return false
      Object next(): this method can fetch data from collection and move cursor on next element
      void remove(): remove data from collection at the time of data fetching

Example with source code

```
package org.techhub;
import java.util.*;
public class VectorApplication {
        public static void main(String[] args) {
                Vector v=new Vector();
                v.add(10);
                v.add(20);
                v.add(30);
                v.add(40);
          Iterator  i=v.iterator();
          while(i.hasNext()) {
                  Object obj =i.next();
                  System.out.println(obj);
          }
        }
}
```

Q. What is the difference between Iterator and Enumeration?

---

a. Iterator can work with legacy as well as non legacy collection means it is universal cursor but Enumeration can work only with a legacy collection

b. Iterator can read data as well as remove data simultaneously from collection but Enumeration is read only cursor means just can read data from collection cannot modify it
c. We can create Iterator reference by using iterator() method and Enumeration reference can use elements()
d. Methods of Iterator boolean hasNext(),Object next() , void remove(),forEachRemaining(), and method of Enumeration hasMoreElements() and nextElements()

4. **Using ListIterator :** ListIterator is a interface from java.util package and which is used for fetch data in forward direction as well as in backward directions
And this interface only works with List collection and it is a child of Iterator means all methods of Iterator present in ListIterator.

Syntax to create reference of ListIterator

ListIterator ref=collref.listIterator(int index): this method can travel collection by using specified index using forward direction fetching and backward direction fetching.

Methods of ListIterator

---

a. **boolean hasNext():** this method check element present in collection or not in forward direction travelling
b. **boolean hasPrevious():** this method check element present in collection or not in backward travelling
c. **Object next():** fetch data from collection and move cursor on next element
d. **Object previous():** fetch data from collection and move cursor on previous element
e. **void add(Object):** this method is used for add new element in collection at the time of travelling
f. **void remove():** this method can remove data from collection at the time of travelling
g. **void set(E):** replace data in collection at the time of travelling
h. **int previousIndex():** this method return index of previous element
i. **int nextIndex():** this method can return index of next element

Example with source code

```
package org.techhub;

import java.util.*;

public class ListIteratorApplication {
	public static void main(String[] args) {
		Vector v = new Vector();
		v.add(10);
```

```
                v.add(20);
                v.add(30);
                v.add(40);
                ListIterator li = v.listIterator(0);
                System.out.println("Data travelling using a forward direction");
                while (li.hasNext()) {
                        Object ele = li.next();
                        System.out.println(ele);
                }
                li = v.listIterator(v.size());
                while (li.hasPrevious()) {
                        Object previous = li.previous();
                        if ((int) previous == 30) {
                                li.add(1000);
                        }
                }
                System.out.println("Data travelling using backward direction");

                li = v.listIterator(v.size());
                while (li.hasPrevious()) {
                        Object previous = li.previous();
                        System.out.println(previous);
                }
        }
}
```

Q. What is the difference between Iterator and ListIterator?

Iterator	**ListIterator**
Iterator can fetch data only using forward direction	ListIterator can fetch data using both direction
Iterator can work with all types of collection so it is known universal cursor	ListIterator can work only with list collection
Iterator can remove data from collection at the time of travelling	ListIterator can add(),replace(),remove() data in collection
Iterator is parent of ListIterator	ListIterator is child of Iterator
Methods of Iterator boolean hasNext() Object next() void remove()	**Methods of ListIterator** Boolean hasPrevious() Object previous() Object next() Boolean hasNext() void add(E) void set(E) void remove()

	int previousIndex() Int nextIndex()

5. **Using enhance for loop or for each:** this concept introduced by java in JDK 1.5 version and normally this is used for fetching data from collection or array.

   Syntax:

   for(datatype variablename:array or collection)
   {
   }

Example with source code

```
package org.techhub;
import java.util.*;
public class EnhanceForApplication {
        public static void main(String[] args) {
                int arr[]=new int[] {10,20,30,40,50};
                for(int val:arr) //for(int i=0;i<arr.length;i++)
                {  System.out.println(val);
                }
        }
}
```

Or

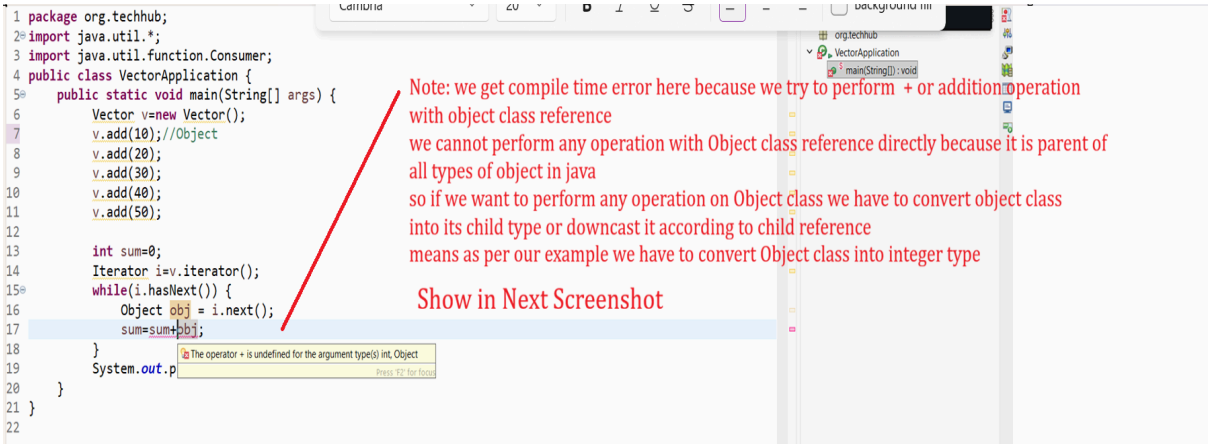
```
package org.techhub;
import java.util.*;
public class VectorApplication {
        public static void main(String[] args) {
                Vector v=new Vector();
                v.add(10);
                v.add(20);
                v.add(30);
                v.add(40);
         for(Object obj:v) {
                System.out.println(obj);
         }
        }
}
```

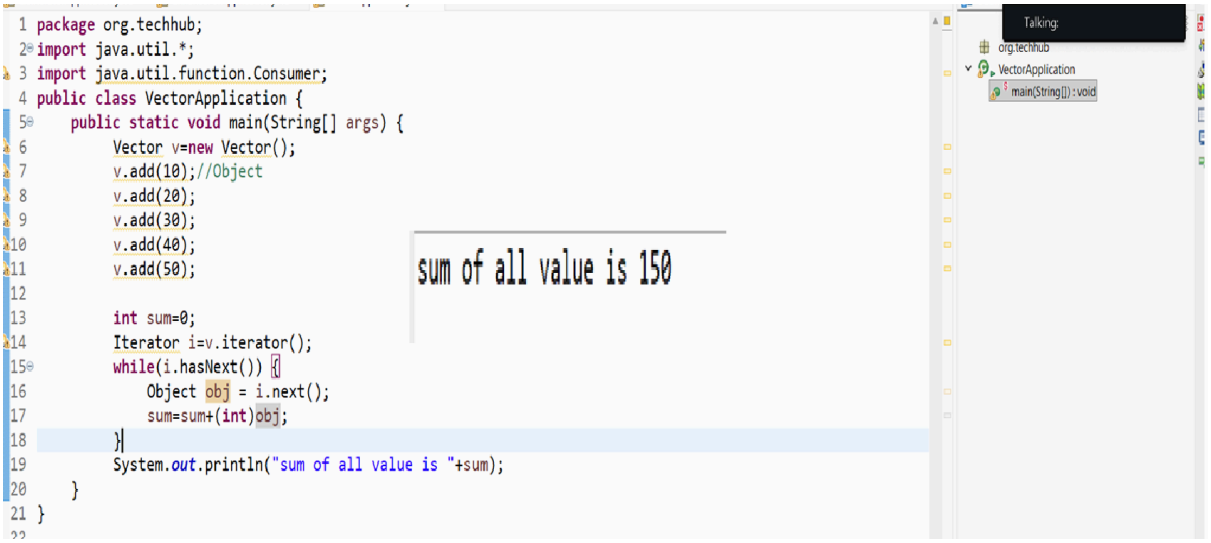
6. **Using forEach() method of JDK 1.8 version**

```
package org.techhub;
import java.util.*;
import java.util.function.Consumer;
public class VectorApplication {
        public static void main(String[] args) {
```

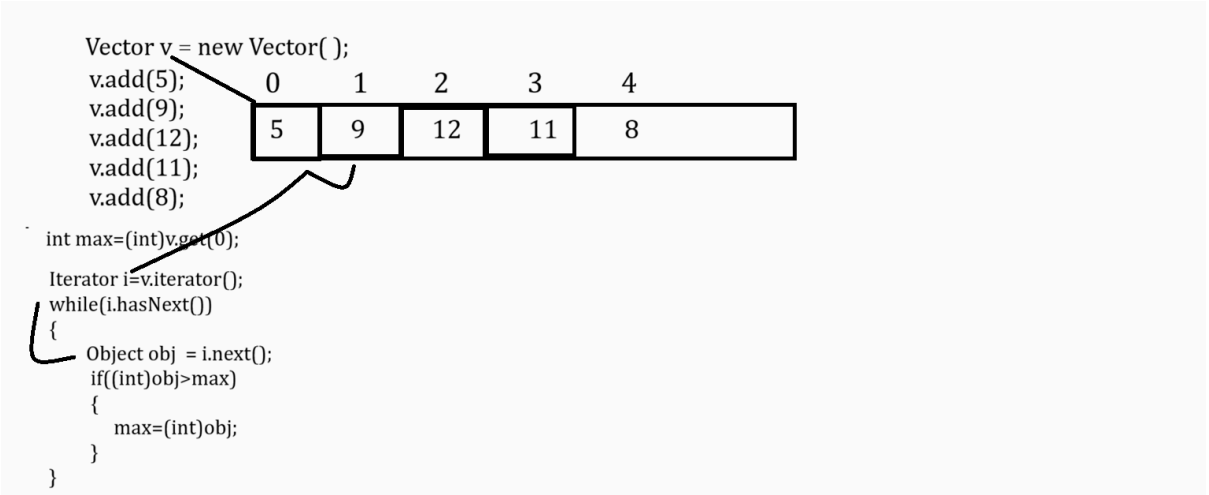
```
                Vector v=new Vector();
                v.add(10);
                v.add(20);
                v.add(30);
                v.add(40);
                v.forEach((t)->System.out.println(t));
            }
        }
```

Example:WAP to create a Vector and store 5 values in it and calculate the sum of all values.





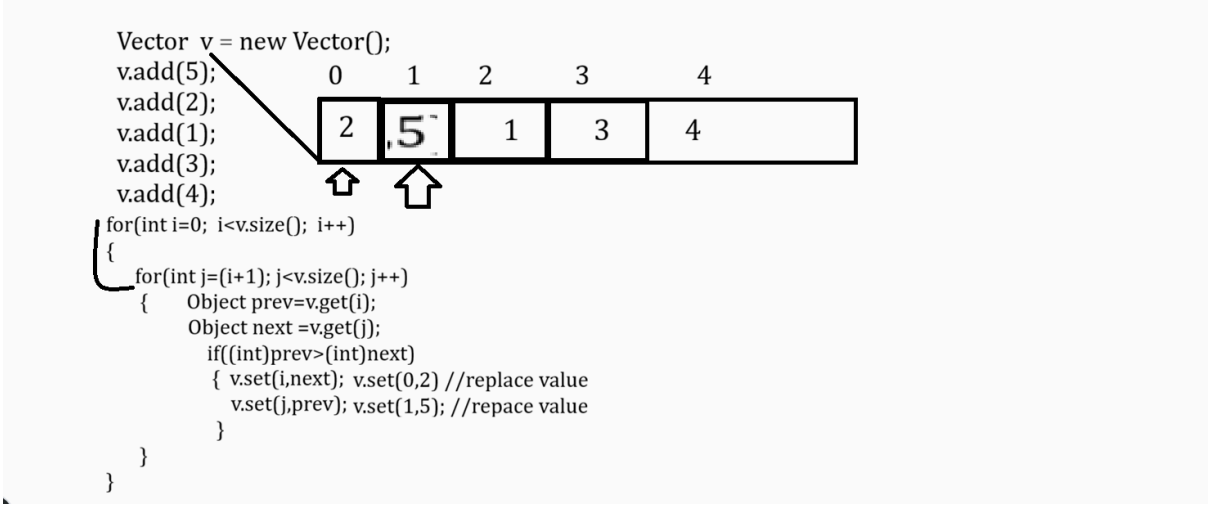
Example: WAP to store 5 values in Vector and find the max value




```java
package org.techhub;
import java.util.*;
import java.util.function.Consumer;
public class VectorApplication {
    public static void main(String[] args) {
        Vector v=new Vector();
        v.add(5);//Object
        v.add(9);
        v.add(12);
        v.add(11);
        v.add(8);
        v.get(0);
        int max=(int)v.get(0);
        Iterator  i=v.iterator();
        while(i.hasNext()) {
            Object obj = i.next();
            if((int)obj>max) {
                max=(int)obj;
            }
        }
        System.out.println("Max value is "+max);
    }
}
```

```
Max value is 12
```

Example: WAP to create Vector with 5 values and sort in ascending order

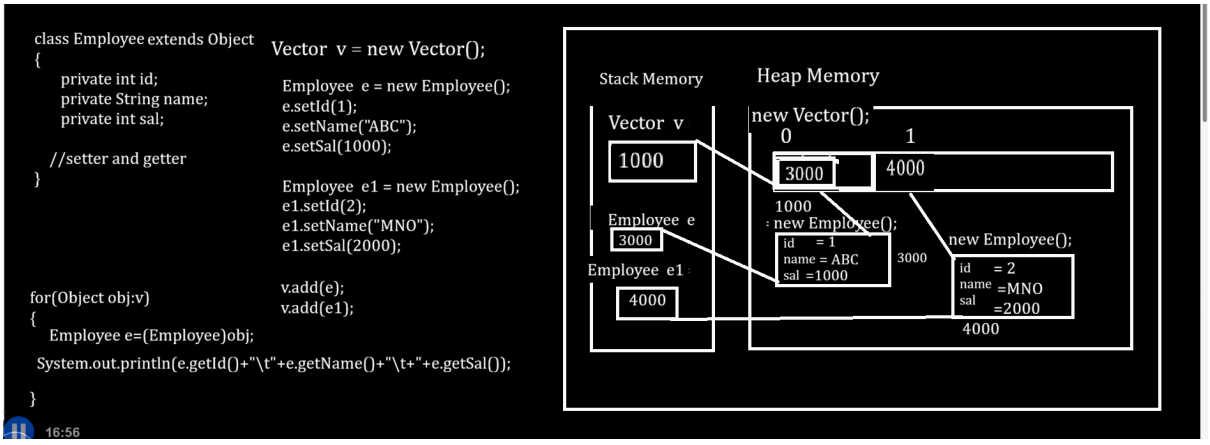


Example with source code

```java
package org.techhub;
import java.util.*;
import java.util.function.Consumer;
public class VectorApplication {
        public static void main(String[] args) {
```

```
                Vector  v=new Vector ();
                v.add(5);//Object
                v.add(9);
                v.add(12);
                v.add(11);
                v.add(8);
        System.out.println("Before sorting");
        for(Object obj:v) {
                 System.out.print(obj+"\t");
        }
                for(int i=0; i<v.size();i++) {
                        for(int j=(i+1);j<v.size();j++) {
                                Object prev=v.get(i);
                                Object next=v.get(j);
                                if((int)prev>(int)next) {
                                        v.set(i, next);
                                        v.set(j, prev);
                                }
                        }
                }
                System.out.println("\nAfter sorting");
                 for(Object obj:v) {
                         System.out.print(obj+"\t");
                 }
        }
}
```

How to store user defined objects in Vector



Example with source code

```
package org.techhub;

import java.util.*;

class Employee {
```

```
	private int id;
	private String name;

	public Employee() {

	}

	public Employee(String name, int id, int sal) {
		this.name = name;
		this.id = id;
		this.sal = sal;
	}

	public int getId() {
		return id;
	}

	public void setId(int id) {
		this.id = id;
	}

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

	public int getSal() {
		return sal;
	}

	public void setSal(int sal) {
		this.sal = sal;
	}

	private int sal;

}

public class EmployeeVectApplication {

	public static void main(String[] args) {
		Vector v = new Vector();

		Employee e = new Employee();
		e.setId(1);
```

```
		e.setName("ABC");
		e.setSal(10000);

		Employee e1 = new Employee();
		e1.setId(2);
		e1.setName("PQR");
		e1.setSal(2000);

		v.add(e);
		v.add(e1);
		System.out.println("Display employee data");
		for (Object obj : v) {
			Employee e2 = (Employee) obj;
			System.out.println(e2.getId() + "\t" + e2.getName() + "\t" + e2.getSal());
		}
	}
}
```

Or

```
package org.techhub;

import java.util.*;

class Employee {
	private int id;
	private String name;

	public Employee() {

	}

	public Employee(String name, int id, int sal) {
		this.name = name;
		this.id = id;
		this.sal = sal;
	}

	public int getId() {
		return id;
	}

	public void setId(int id) {
		this.id = id;
	}

	public String getName() {
		return name;
	}
```

```
	public void setName(String name) {
		this.name = name;
	}

	public int getSal() {
		return sal;
	}

	public void setSal(int sal) {
		this.sal = sal;
	}

	private int sal;

}

public class EmployeeVectApplication {

	public static void main(String[] args) {
		Vector v = new Vector();

		Employee e = new Employee("ABC",1,1000);
		Employee e1 = new Employee("MNO",2,2000);

		v.add(e);
		v.add(e1);
		System.out.println("Display employee data");
		for (Object obj : v) {
			Employee e2 = (Employee) obj;
			System.out.println(e2.getId() + "\t" + e2.getName() + "\t" + e2.getSal());
		}
	}
}
```

Or

```
package org.techhub;
import java.util.*;
class Employee {
	private int id;
	private String name;
	public Employee() {
	}
	public Employee(String name, int id, int sal) {
		this.name = name;
		this.id = id;
		this.sal = sal;
	}
	public int getId() {
```

```
		return id;
	}
	public void setId(int id) {
		this.id = id;
	}
	public String getName() {
		return name;
	}
	public void setName(String name) {
		this.name = name;
	}
	public int getSal() {
		return sal;
	}
	public void setSal(int sal) {
		this.sal = sal;
	}
	private int sal;
}
public class EmployeeVectApplication {
	public static void main(String[] args) {
		Vector v = new Vector();
		v.add(new Employee("ABC",1,1000));
		v.add(new Employee("MNO",2,2000));
		System.out.println("Display employee data");
		for (Object obj : v) {
			Employee e2 = (Employee) obj;
			System.out.println(e2.getId() + "\t" + e2.getName() + "\t" +
e2.getSal());
		}
	}
}
```

Example: WAP to create Vector and store player class objects in it and perform following operation on it

Case 1: Add New Player
Case 2: View All Player
Case 3: Delete Player by id
Case 4: count number of player in collection
Case 5: Search player by id or name

Example with source code

```
package org.techhub;
import java.util.*;
class Player {
	private int id;
	private String name;
```

```
	public Player() {
	}
	public Player(String name, int id, int run) {
		this.name = name;
		this.id = id;
		this.run = run;
	}
	public int getId() {
		return id;
	}
	public void setId(int id) {
		this.id = id;
	}
	public String getName() {
		return name;
	}
	public void setName(String name) {
		this.name = name;
	}
	public int getRun() {
		return run;
	}
	public void setRun(int run) {
		this.run = run;
	}
	private int run;
}
public class VectorPlayerApplication {
	public static void main(String[] args) {
		Vector v = new Vector();
		do {
			Scanner xyz = new Scanner(System.in);
			System.out.println("1:Add New Employee");
			System.out.println("2:View All Employee");
			System.out.println("3:Delete player by id");
			System.out.println("4:Count number of player");
			System.out.println("5:Search Player by id");
			System.out.println("Enter your choice");
			int choice = xyz.nextInt();
			switch (choice) {
			case 1:
				xyz.nextLine();
				System.out.println("Enter name id and salary of player");
				String name = xyz.nextLine();
				int pid = xyz.nextInt();
				int run = xyz.nextInt();
				Player p = new Player(name, pid, run);
				boolean b = v.add(p);
```

```
            if (b) {
            System.out.println("New Player added successfully.....");
            } else {
                    System.out.println("Some problem is there.....");
            }
            break;
    case 2:
            if (v.size() != 0) { // indicate player data present in vector
                    for (Object obj : v) {
                            p = (Player) obj;
    System.out.println(p.getId() + "\t" + p.getName() + "\t" + p.getRun());
                    }
            } else {
    System.out.println("No player data present in collection or vector");
            }
            break;
    case 3:
            if (v.isEmpty()) {
                    System.out.println("Collection is empty");
            } else {
                    System.out.println("Enter player id for delete");
                    int playerId = xyz.nextInt();
                    int index = -1;
                    Iterator i=v.iterator();
                    while(i.hasNext()) {
                            Object obj=i.next();
                            Player p1=(Player)obj;
                            if(p1.getId()==playerId) {
                                    index=v.indexOf(p1);
                                    i.remove();//remove using a iterator
                            }
                    }
                    if (index != -1) {
            System.out.println("Player deleted successfully...");
                    } else {
                            System.out.println("Player not deleted");
                    }
            }
            break;
    case 4:
            System.out.println("Number of player in vector "+v.size());
            break;
    case 5:
            System.out.println("Enter player id for search");
            int playerId = xyz.nextInt();
            int index = -1;
            for (Object obj : v) {
                    Player p1 = (Player) obj;
```

```
                                        if (p1.getId() == playerId && v.indexOf(p1) != -1) {
                                                index=v.indexOf(p1);

                                        }
                                }
                                if (index != -1) {
                                        System.out.println("Player found successfully...");
                                } else {
                                        System.out.println("Player not  found");
                                }
                                break;
                        default:
                                System.out.println("Wrong choice");
                        }
                } while (true);// infinite loop
        }
}
```

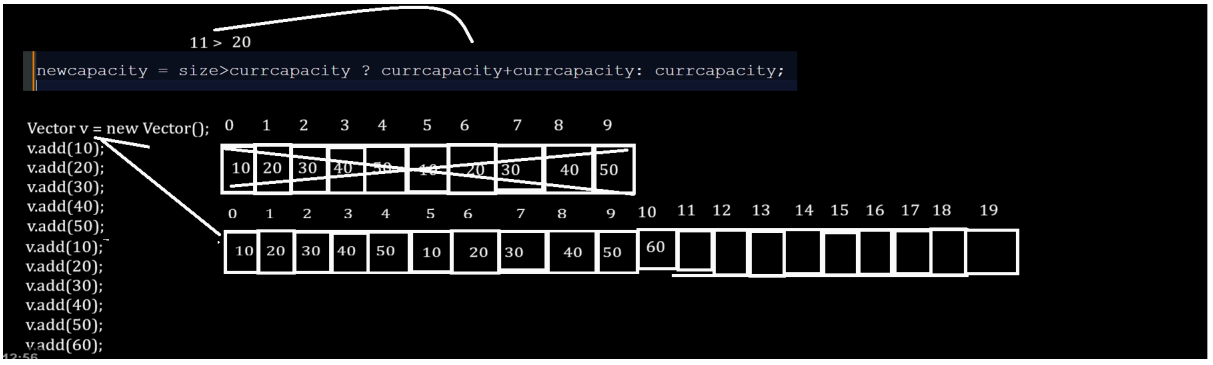
ArrayList:

ArrayList is a same like as Vector with some major difference or ArrayList is also dynamic array internally

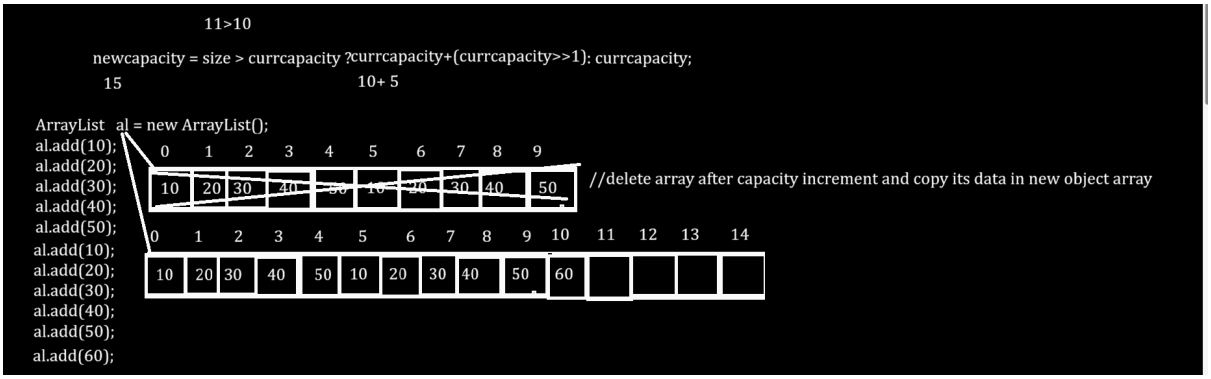
Q. What is the difference between Vector and ArrayList?

---

1. Vector is legacy collection and ArrayList is non legacy collection
2. Vector occupy double memory than its current capacity and ArrayList occupy half memory than its current capacity
3. Vector is thread safe or synchronized collection but ArrayList is an asynchronous collection or non thread safe collection
4. Vector is slower than ArrayList
5. Vector use following logic for capacity increment



ArrayList use following logic for capacity increment



Q. What are the similarities between Vector and ArrayList?

---

1. Both work as dynamic array
2. Both are implementer classes of List interface
3. Default capacity of Vector and ArrayList is 10

Note: Vector is recommended in thread safe environment and ArrayList is recommended in non -thread safe environment

Example: Write program to store 5 values in ArrayList and display it

```
package org.techhub;
import java.util.*;
public class ArrayListApplication {
	public static void main(String[] args) {
	  ArrayList al = new ArrayList();
	  al.add(10);
	  al.add(20);
	  al.add(30);
	  al.add(40);
	  al.add(50);
	  for(Object obj:al) {
		System.out.println(obj);
	  }
	}
}
```

Example: WAP to create Product class with field id,name and price and store 5 product details in ArrayList and display it

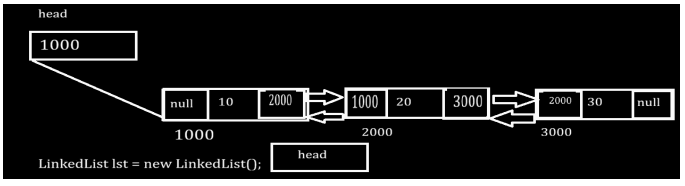
```
package org.techhub;

public class Product {
	private int id;
	private String name;
	public Product() {

	}
```

```
	public Product(String name,int id,int price) {
		this.name=name;
		this.id=id;
		this.price=price;
	}
	public int getId() {
		return id;
	}
	public void setId(int id) {
		this.id = id;
	}
	public String getName() {
		return name;
	}
	public void setName(String name) {
		this.name = name;
	}
	public int getPrice() {
		return price;
	}
	public void setPrice(int price) {
		this.price = price;
	}
	private int price;
}
package org.techhub;
import java.util.*;
public class ArrayListApplication {
	public static void main(String[] args) {
		ArrayList al = new ArrayList();
		al.add(new Product("Mouse", 1, 1000));
		al.add(new Product("Keyboard", 2, 1000));
		al.add(new Product("Laptop", 3, 100000));
		al.add(new Product("Remote", 4, 1000));
		System.out.println("Display product list");
		for (Object obj : al) {
			Product p = (Product) obj;
	System.out.println(p.getId() + "\t" + p.getName() + "\t" + p.getPrice());
		}
	}
}
```

LinkedList: LinkedList in java is by default Double LinkedList and it is inbuilt class provided by java to us.



Q. What is the difference between ArrayList and LinkedList?

---

1. Internally ArrayList is a dynamic array and LinkedList is by default a double linked list.
2. ArrayList use less memory than linked list and LinkedList required more memory than ArrayList because pointers
3. ArrayList is faster for data retrieval and LikedList is slower because in ArrayList we fetch using base address+index*size but in LinkedList we need to find the address of next node or previous node i.e memory address and memory address allocation is not sequential so it will take some time for data fetching
   So time complexity of ArrayList for fetching data is O(1) and LinkedList has O(n)
4. If we want to insert data in middle of ArrayList then it required time complexity O(n) and LinkedList O(1)
5. If we want to delete data from middle in ArrayList then it required shifting element at lower side index by 1 so the time complexity is O(n) but if we think about LinkedList not required element shifting so time complexity is O(1)
6. ArrayList is recommended for data fetching and searching but LinkedList is recommended for insertion and deletion

Example with source code

```
package org.techhub;
import java.util.*;
public class LinkedListApplication {
	public static void main(String[] args) {

			LinkedList lst = new LinkedList();
			lst.add(10);
			lst.add(20);
			lst.add(30);
			lst.add(40);
			lst.add(50);
			Iterator i=lst.iterator();
			while(i.hasNext()) {
				Object obj =i.next();
				System.out.println(obj);
			}

	}
}
```

Example of LinkedList with user defined objects

```
package org.techhub;
import java.util.*;
class Product{
	private int id;
	public int getId() {
		return id;
	}
```

```
	public void setId(int id) {
		this.id = id;
	}
	public String getName() {
		return name;
	}
	public void setName(String name) {
		this.name = name;
	}
	public int getPrice() {
		return price;
	}
	public void setPrice(int price) {
		this.price = price;
	}
	private String name;
	private int price;

	public Product() {

	}
	public Product(int id,String name,int price) {
		this.id=id;
		this.name=name;
		this.price=price;
	}
}
public class LinkedListApplication {
	public static void main(String[] args) {
			LinkedList lst = new LinkedList();
			 Product p1 = new Product(1,"ABC",100);
			 Product p2 = new Product(2,"XYZ",200);
			 Product p3 = new Product(3,"STV",300);
			 lst.add(p1);
			 lst.add(p2);
			 lst.add(p3);
			 Iterator i = lst.iterator();
			 while(i.hasNext()) {
				Object object=i.next();
				Product p=(Product)object;

System.out.println(p.getId()+"\t"+p.getName()+"\t"+p.getPrice());
			 }
	}
}
```

Stack: Stack is inbuilt class in java and it is implementer class of List collection and manage data in first in last out format and it is direct child of Vector class

Methods of Stack Collection

---

public E push(E item): this method is used for push element in stack and increase top by 1
public synchronized E pop(): this method is used for remove the top most data from stack which we added last
public synchronized E peek(): this method is used for fetch only top element but not remove just retrieve it
public boolean isEmpty(): this method is used for check stack is empty or not if empty return true otherwise return false
public synchronized int search(Object o): this method is used for search data from stack and return its index with difference from top or return element different from top and if element not found return -1

Example with source code

```
package org.techhub;

import java.util.*;

public class StackApplication {
	public static void main(String[] args) {
		Stack s = new Stack();
		Scanner xyz = new Scanner(System.in);
		do {
			System.out.println("1: Insert Data in Stack");
			System.out.println("2: Delete data from stack");
			System.out.println("3: Search data from stack");
			System.out.println("4: View top most data");
			System.out.println("5: View All data from stack");
			System.out.println("Enter your choice");
			int choice = xyz.nextInt();
			switch (choice) {
			case 1:
				System.out.println("Enter data in stack");
				int data = xyz.nextInt();
				s.push(data);
				break;
			case 2:
				if (s.isEmpty()) {
					System.out.println("Stack is empty");
				} else {
					Object obj = s.pop();
					System.out.println("Deleted data is " + obj);
				}
				break;
```

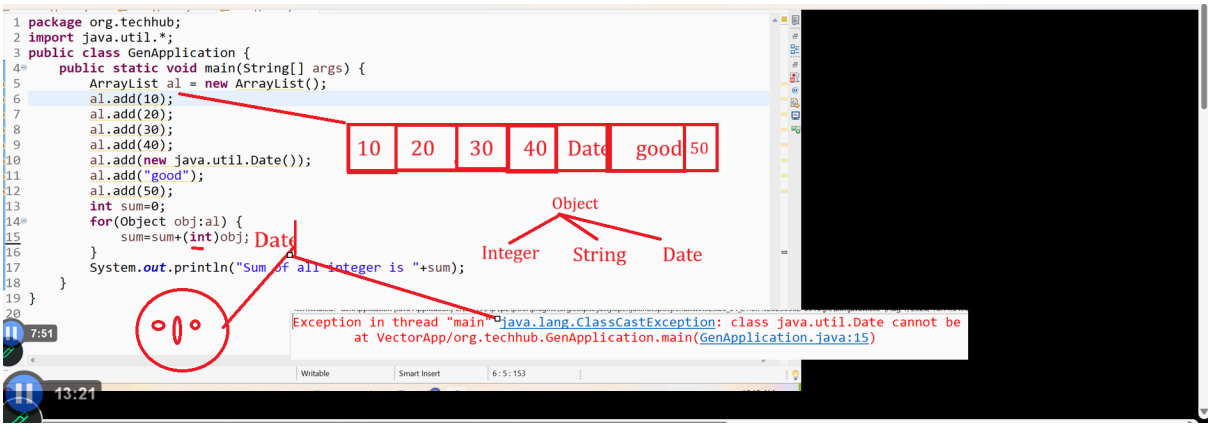
```
                case 3:
                        if (s.isEmpty()) {
                                System.out.println("Stack is empty");
                        } else {
                                System.out.println("Enter search data");
                                int sdata = xyz.nextInt();
                                int index = s.search(sdata);
                                if (index != -1) {
                                        System.out.println("element found");
                                } else {
                                        System.out.println("element not found");
                                }
                        }
                        break;
                case 4:
                        if (s.isEmpty()) {
                                System.out.println("Stack is empty");
                        } else {
                                Object obj = s.peek();
                                System.out.println("Your Top Most Data " + obj);
                        }
                        break;
                case 5:
                        ListIterator li=s.listIterator(s.size());
                        while(li.hasPrevious()) {
                                Object obj = li.previous();
                                System.out.println(obj);
                        }
                        break;
                default:
                        System.out.println("Wrong choice");
                }
        } while (true);

    }
}
```

Generics Concept in JAVA?

Generic is a concept in java which is used for avoid the ClassCastException at runtime specially working with collection Framework
ClassCastException occur at runtime when we try to perform casting between two objects Or two references normally ClassCastException occur in upcasting or in downcasting also If we think about Collection Framework. Collection can store data in the form of Object class And Object class can store any kind of object or data but we cannot perform Operation on Object class and if we try to Object class to specific class using downcasting technique then There is possibility we get ClassCastException so avoiding this type of situation we can use Generics with concept

So Generic provide compile time safety to collection framework



If we want to solve this problem we have two solutions

1. **Instanceof operator :** instanceof operator is used to compare an object means instanceof operator return true if object belongs from a particular class otherwise return false.

   Example with source code

```
package org.techhub;
import java.util.*;
public class GenApplication {
        public static void main(String[] args) {
                ArrayList al = new ArrayList();
                al.add(10);
                al.add(20);
                al.add(30);
                al.add(40);
                al.add(new java.util.Date());
                al.add("good");
                al.add(50);
                int sum = 0;
                for (Object obj : al) {
                        if (obj instanceof Integer) {
                                sum = sum + (int) obj;
                        }
                }
                System.out.println("Sum of all integer is " + sum);
        }
}
```

   Instance of operator not provide compile time safety you have to check object with type at runtime but if we want to provide compile time safety and apply restriction on collection to work with specific type then we can use generics

   If we think about above example we want to our ArrayList object al must be work with integer data and when developers try to store any other type of data in ArrayList using

al object then al should not accept it and compiler should generate compile time error to us in this case we can generics means we provide compile time restriction on collection to work with a specific

2. **Generics :** generics is denoted by <> shown in following examples

Example with source code

```
package org.techhub;
import java.util.*;
public class GenApplication {
	public static void main(String[] args) {
		ArrayList <Integer>al = new ArrayList<Integer>();
		al.add(10);
		al.add(20);
		al.add(30);
		al.add(40);
		al.add(50);
		int sum = 0;
		for(Integer val:al) {
			sum=sum+val;
		}
		System.out.println("Sum of all integer is " + sum);
	}
}
```

Or

We can fetch data using Iterator like as

```
package org.techhub;
import java.util.*;
public class GenApplication {
	public static void main(String[] args) {
		ArrayList <Integer>al = new ArrayList<Integer>();
		al.add(10);
		al.add(20);
		al.add(30);
		al.add(40);
		al.add(50);
		int sum = 0;
		Iterator  <Integer>i=al.iterator();
		while(i.hasNext()) {
			Integer val=i.next();
			sum=sum+val;
		}
		System.out.println("Sum of all integer is " + sum);
	}
}
```

Note: if we think about above example we loose main feature of Collection store any kind of data because above collection store only integer

How we can store different types of data using Generics and Avoid ClassCastException

---

If we want to store different types of data using a Generics we can use POJO class concept means we store all data or all types of Data in Pojo class and use POJO as generic type with collection framework
So when we fetch data then collection return POJO class and we can fetch specified data from Pojo class object

Example: suppose consider we want to store five product details in Collection and calculate sum of all products price and Product information like as id,name , price so price should be float ,id should be integer and name should be string

```
package org.techhub;
import java.util.*;
class Product {
	private int id;
	private String name;

	public Product() {

	}

	public Product(String name, int id, float price) {
		this.name = name;
		this.id = id;
		this.price = price;
	}
	public int getId() {
		return id;
	}
	public void setId(int id) {
		this.id = id;
	}
	public String getName() {
		return name;
	}
	public void setName(String name) {
		this.name = name;
	}

	public float getPrice() {
		return price;
	}
	public void setPrice(float price) {
		this.price = price;
	}
```

```
	private float price;
}
public class GenApplication {
	public static void main(String[] args) {
		ArrayList<Product> al = new ArrayList<Product>();
		al.add(new Product("ABC", 1, 1000.0f));
		al.add(new Product("MNO", 2, 2000.0f));
		al.add(new Product("PQR", 3, 3000.0f));
		float sum = 0;
		for (Product p : al) {
			sum = sum + p.getPrice();
		}
		System.out.println("Sum of all product price " + sum);

	}
}
```

Types of Generics

If we want to work with types of Generics we have some standard generic type given by JAVA to us

a. **E :** E stands for generic elements
b. **T :** T stands for generic type
c. **K** : K stands for Generic Key
d. **V** : V stands for Generic Value

Now we want to types of Generics

---

a. Class with Generics
b. Interface with Generics
c. Wild card generics
    1. Bounded Generics
       I. upper bound generics
       II. Lower Bounds generics
    2. Unbounded generics

Class with Generics : Class with generic is facility provided by Java to us User can create own class as Generic class

Syntax of user defined class as generic class

```
public class classname<E>{
}
```

Example with source code

```
package org.techhub;
class Test<E>{
```

```
	void add(E x) {
		System.out.println("X is "+x);
	}
}
public class TestUserClassAsGenApplication {
	public static void main(String[] args) {
		 Test <Integer>t= new Test<Integer>();
		 t.add(100);
		 t.add(200);
		 t.add(300);
		 t.add(400);
		 t.add(500);
	}
}
```

Interface with Generic concept : Interface with generics help us we can modify the interface method in every implementer class according to his requirements called as interface with generics

Syntax:

```
interface interfacename<T>
{
}
```

Example:

```
package org.techhub;
interface Circle<T>{
	  void setRadius(T r);
	  T getResult();
}
class Area<E> implements Circle<Integer>{

	@Override
	public void setRadius(Integer r) {
		// TODO Auto-generated method stub

	}

	@Override
	public Integer getResult() {
		// TODO Auto-generated method stub
		return null;
	}



}
class Cirm<E> implements Circle<Float>{

	@Override
```

```
	public void setRadius(Float r) {
		// TODO Auto-generated method stub

	}

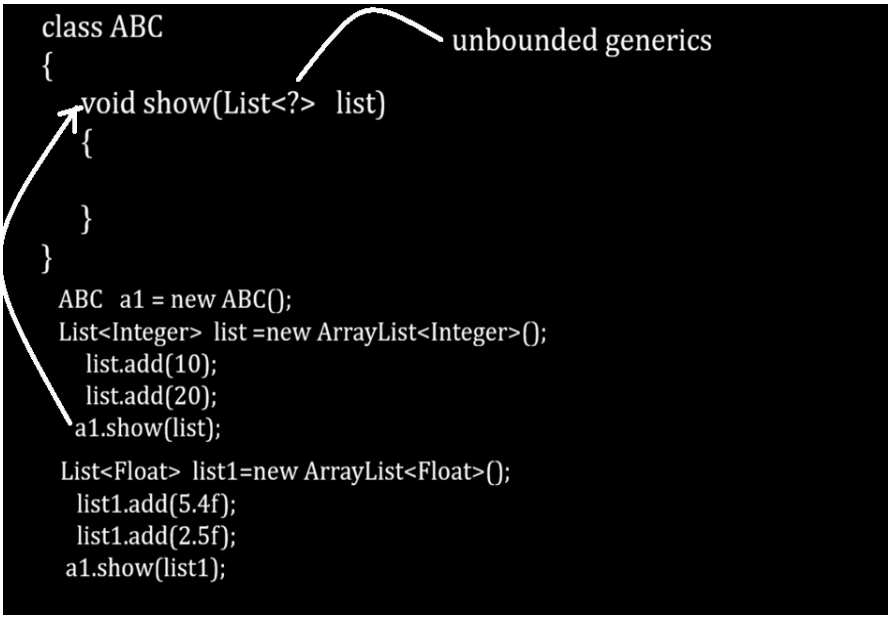
	@Override
	public Float getResult() {
		// TODO Auto-generated method stub
		return null;
	}
}
public class InterfaceGenApplication {
	public static void main(String[] args) {
		// TODO Auto-generated method stub
	}
}
```

Wild Card Generics: wild card generics is denoted by ? the goal of wild card generics is to provide flexibility with type using a generics

There are two types of wild card generics

---

a. **Unbounded generics :** basically unbounded generics denoted using only ? and we use it pass parameter in method e.g use with collection like as



Example with source code

```
package org.techhub;
import java.util.*;
class ABC
{
	void show(List<?> list)
```

```
        {
                for(Object obj:list) {
                        System.out.println(obj);
                }
        }
}
public class GenApplication {
        public static void main(String[] args) {
          ABC a1 = new ABC();
          System.out.println("Generics with integer value");
          List<Integer> list= new ArrayList<Integer>();
          list.add(10);
          list.add(20);
          list.add(30);
          a1.show(list);
          System.out.println("Generics with float value");
          List<Float> list1= new ArrayList<Float>();
          list1.add(5.4f);
          list1.add(2.4f);
          list1.add(3.5f);
          a1.show(list1);
        }
}
```

b. **Bounded Generics:** bounded generics normally flexible with parent and child datatypes

There are two types of bounded generics

1. **Upper bound generics:** An upper bound generics means allow class and its sub classes called as upper bound generics
   Note: it is denoted by using extends keyword
   Syntax: <? extends classname>

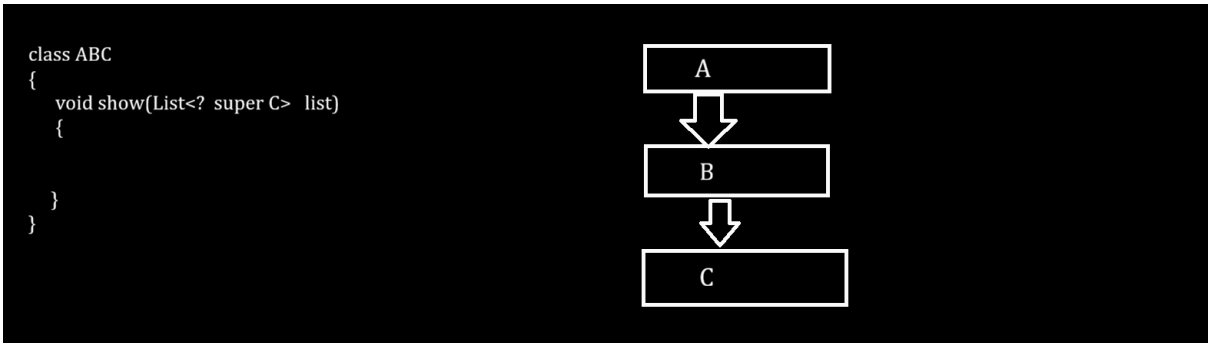
Example with source code

```
package org.techhub;
import java.util.*;
class ABC
{   double sum=0;
        void show(List<? extends Number> list)
        {       for(Number val:list) {
                        if(val instanceof Integer)
                          sum=sum+(Integer)val;
                        if(val instanceof Float) {
                                sum=sum+(Float)val;
                        }
                 }
                 System.out.println("Sum is "+sum);
```

```
        }
}
public class GenApplication {
        public static void main(String[] args) {
          ABC a1 = new ABC();
          System.out.println("Generics with integer value");
          List<Integer> list= new ArrayList<Integer>();
          list.add(10);
          list.add(20);
          list.add(30);
          a1.show(list);
          System.out.println("Generics with float value");
          List<Float> list1= new ArrayList<Float>();
          list1.add(5.4f);
          list1.add(2.4f);
          list1.add(3.5f);
          a1.show(list1);
        }
}
```

2. **Lower bound generics :** Lower bounded generics means allow class and its super class with generic type called as lower bound generics and it is denoted by using super keyword

Syntax: <? super classname>



Example with source code

```
package org.techhub;
import java.util.*;
class A
{  void showA() {
                System.out.println("I am show A method");
        }
}
class B extends A
{   void showB() {
                System.out.println("I am show B method");
        }
```

```
}
class C extends B
{   void showC() {
                System.out.println("I am Show C method");
        }
}
class Test{
        void show(List<? super C> list) {
                for(Object c:list) {
                        if(c instanceof A) {
                                ((A) c).showA();
                        }
                        if(c instanceof B) {
                                ((B) c).showB();
                        }
                }
        }
}
public class LowerBoundApp {
        public static void main(String[] args) {
                List<A> list = new ArrayList<A>();
                 list.add(new A());
                 Test  t = new Test();
                 t.show(list); //List<A super C>

                 List<B> list1=new ArrayList<B>();
                 list1.add(new B());
                 t.show(list1);// List<B super C>
        }
}
```

Collections class

Collections is a utility class of java which is used for perform regular data structure operation on collection framework like as sort collection , finding max value from collection , finding minimum value from collection etc
Collections class contain only static method which help us to perform Data structure operation on Collection Framework

Methods of Collections class

a. **public static void sort(List<T>):** this method can sort primitive type of collection as well as user defined object using a single field with the help of Comparable interface
b. **public static void sort(List<T>,Comparator):**sort user defined object collection using a multiple field
c. **public static Object max(Collection<T>):** this method is used for find the max value from collection

d. **public static Object min(Collection<T> list):** this method is used for find the min value from collection
e. **public static void reverse(List<T>):** this method is used for reverse the list collection

Example with source code

```
import java.util.*;
public class GenApplication {
	public static void main(String[] args) {
	    List<Integer> list=new ArrayList<Integer>();
	    list.add(5);
	    list.add(2);
	    list.add(3);
	    list.add(1);
	    list.add(4);
	    System.out.println("Collection without sorting");
	    for(Object obj:list) {
		System.out.print(obj+"\t");
	    }
		Object objMax =Collections.max(list);
		Object objMin=Collections.min(list);
		System.out.println("\nMax value is "+objMax);
		System.out.println("\nMin value is  "+objMin);
	    Collections.sort(list);
	    System.out.println("\nCollection with sorting");
	    for(Object obj:list) {
		System.out.print(obj+"\t");
	    }
		Collections.reverse(list);
		 System.out.println("\nCollection with reverse");
	    for(Object obj:list) {
		System.out.print(obj+"\t");
	    }
	}
}
```

Example: WAP TO Create Employee class with id,name and sal and store 5 employee objects in Collection and sort it using Collections class

Example with source code

```
package org.techhub;
import java.util.*;
class Employee {
	private int id;
	private String name;
```

```
	public Employee() {

	}
	public Employee(String name,int id,int sal) {
		this.name=name;
		this.id=id;
		this.sal=sal;
	}
	public int getId() {
		return id;
	}

	public void setId(int id) {
		this.id = id;
	}

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

	public int getSal() {
		return sal;
	}

	public void setSal(int sal) {
		this.sal = sal;
	}

	private int sal;

}

public class EmployeeSortingApplication {
	public static void main(String[] args) {
		List<Employee> al = new ArrayList<Employee>();
		al.add(new Employee("ABC",1,10000));
		al.add(new Employee("MNO",5,2000));
		al.add(new Employee("PQR",2,20000));
		al.add(new Employee("STV",3,3000));
		al.add(new Employee("OOO",4,4000));
		System.out.println("Display employee data before sorting");
		for(Employee e:al) {
			System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
		}
```

```
		Collections.sort(al);
		System.out.println("Display employee data before sorting");
		for(Employee e:al) {
			System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
		}
	}
}
```

Note: if we think about above code we get compile time error because when we have list collection with a user define objects then Collections.sort() method cannot sort data because if we think about user define objects it contain different field e.g suppose if we think about Employee class it contain id,name and sal variables so Collections.sort() method cannot predicate using which field data should be sort

If we want to solve this problem we have two interfaces

a. **Comparable :** Comparable is interface which is used for sort the user define objects using a Collections.sort() method by using single field
b. **Comparator :** Comparator is an interface which is used for sort the user define objects using a Collections.sort() method by using multiple field.

Steps to work Comparable interface

---

a. **Add java.lang package in application**
   import java.lang.*;
   Note: no need to import java.lang package because it is the default package of java.
b. **Create user define class and implements Comparable interface in it**

```
package org.techhub;
import java.util.*;
class Employee implements Comparable<Employee>{
	private int id;
	private String name;
	public Employee() {
	}
	public Employee(String name,int id,int sal) {
		this.name=name;
		this.id=id;
		this.sal=sal;
	}
	public int getId() {
		return id;
	}
	public void setId(int id) {
		this.id = id;
	}
	public String getName() {
		return name;
	}
	public void setName(String name) {
		this.name = name;
```

```
	}
	public int getSal() {
		return sal;
	}

	public void setSal(int sal) {
		this.sal = sal;
	}
	private int sal;
}
```

c. Override its method name as int compareTo(T o):

This method is used for write the sorting logics by using a specified field

But if we want to work with int compareTo() method we have some following rules

1. If working object or current is greater than parameter object then return 1
2. If working object or current object is less than parameter object then return -1
3. If working object is equal with current object then return 0

Example: we want to sort employee data by using employee id

```
package org.techhub;

import java.util.*;

class Employee implements Comparable<Employee> {
	private int id;
	private String name;

	public Employee() {
	}

	public Employee(String name, int id, int sal) {
		this.name = name;
		this.id = id;
		this.sal = sal;
	}

	public int getId() {
		return id;
	}

	public void setId(int id) {
		this.id = id;
	}

	public String getName() {
		return name;
	}
```

```
	public void setName(String name) {
		this.name = name;
	}

	public int getSal() {
		return sal;
	}

	public void setSal(int sal) {
		this.sal = sal;
	}

	private int sal;

	@Override
	public int compareTo(Employee o) {

		if (this.id > o.id) {
			return 1;
		} else if (this.id < o.id) {
			return -1;
		} else {
			return 0;
		}
	}
}

public class EmployeeSortingApplication {
	public static void main(String[] args) {
		List<Employee> al = new ArrayList<Employee>();
		al.add(new Employee("ABC", 1, 10000));
		al.add(new Employee("MNO", 5, 2000));
		al.add(new Employee("PQR", 2, 20000));
		al.add(new Employee("STV", 3, 3000));
		al.add(new Employee("OOO", 4, 4000));
		System.out.println("Display employee data before sorting");
		for (Employee e : al) {
			System.out.println(e.getId() + "\t" + e.getName() + "\t" + e.getSal());
		}
		Collections.sort(al);
		System.out.println("Display employee data before sorting");
		for (Employee e : al) {
			System.out.println(e.getId() + "\t" + e.getName() + "\t" + e.getSal());
		}
	}
}
```

Example: WAP to create class name as Player with field id,name and run and store 5 player data in collection and sort player data by using run.

```
package org.techhub;

import java.util.*;

class Player implements Comparable<Player> {
	private int id;
	private String name;

	public Player() {

	}

	public Player(String name, int id, int run) {
		this.name = name;
		this.run = run;
		this.id = id;
	}

	public int getId() {
		return id;
	}

	public void setId(int id) {
		this.id = id;
	}

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

	public int getRun() {
		return run;
	}

	public void setRun(int run) {
		this.run = run;
	}

	private int run;

	@Override
```

```
	public int compareTo(Player o) {
		return this.run > o.run?1:this.run < o.run?-1:0;
	}
}

public class PlayerApplication {
	public static void main(String[] args) {
		List<Player> list = new ArrayList<Player>();
		list.add(new Player("ABC", 1, 10000));
		list.add(new Player("MNO", 2, 4000));
		list.add(new Player("PQR", 3, 3000));
		list.add(new Player("STV", 4, 1000));
		list.add(new Player("XXXX", 5, 8000));
		System.out.println("Display collection without sorting");
		Iterator<Player> i = list.iterator();
		while (i.hasNext()) {
			Player p = i.next();
			System.out.println(p.getId() + "\t" + p.getName() + "\t" + p.getRun());
		}
		Collections.sort(list);
		System.out.println("Display collection with sorting");
	    i = list.iterator();
		while (i.hasNext()) {
			Player p = i.next();
			System.out.println(p.getId() + "\t" + p.getName() + "\t" + p.getRun());
		}
	}
}
```

Note: The limitation of Comparable interface is perform sorting using a single field
But suppose we want to sort data using different field like as id,name,and run then Comparable is not suitable for this type of requirement so we can use Comparator interface

Comparator interface

Comparator interface is same like as Comparable interface but the difference is Comparable interface can sort data using single field and Comparator interface can sort data by using multiple field
Comparator interface contain compare() method and this method is used for writing sorting logic and this method contain two parameters of POJO class objects

Syntax: int compare(Object,Object): this method can two parameters of employee type object

Rules of compare method

---

a. If first object value is greater than second object value then return 1
b. If first object value is less than second object value then return -1
c. If the first object value is equal with the second object value then return 0.

Steps to use the Comparator interface

a. **Create POJO class or class with a setter and getter**

```
package org.techhub;
public class Employee {
	private int id;
	private String name;
	public int getId() {
		return id;
	}
	public void setId(int id) {
		this.id = id;
	}
	public String getName() {
		return name;
	}
	public void setName(String name) {
		this.name = name;
	}
	public int getSal() {
		return sal;
	}
	public void setSal(int sal) {
		this.sal = sal;
	}
	private int sal;
}
```

b. **Create different classes for every field like as Id,name etc**

Means here if we want to sort data by using id we have create separate class and implement Comparator in it and override its compare() method and write sorting logics , if we want to sort data by using run we have to create separate class for run and implement Comparator interface and override its compare() method and write its logics

Example:

ID sorting logic class

```
package org.techhub;

import java.util.Comparator;

public class SortById implements Comparator<Employee> {
```

```
	@Override
	public int compare(Employee o1, Employee o2) {
		if(o1.getId()>o2.getId()) {
			return 1;
		}
		else if(o1.getId()<o2.getId()) {
			return -1;
		}
		else {
			return 0;
		}
	}

}
```

Salary Sorting logic class

Salary sorting logic class

```
package org.techhub;
import java.util.Comparator;
public class SortBySal implements Comparator<Employee> {
	@Override
	public int compare(Employee o1, Employee o2) {
		if (o1.getSal() > o2.getSal()) {
			return 1;
		} else if (o1.getSal() < o2.getSal()) {
			return -1;
		} else {
			return 0;
		}
	}
}
```

c. Store all POJO class objects in collection

```
package org.techhub;
import java.util.*;
public class ComparatorApplication {
	public static void main(String[] args) {
		List<Employee> list= new ArrayList<Employee>();
		list.add(new Employee("A",3,3000));
		list.add(new Employee("B",4,1000));
		list.add(new Employee("C",2,2000));
		list.add(new Employee("D",1,5000));
		list.add(new Employee("E",4,4000));
		System.out.println("Show Employee Data");
```

```
                for(Employee e:list) {
                        System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
                }
        }

}
```

d. Use following Syntax of Collections.sort() method

Syntax: Collections.sort(List,Comparator):
List: this parameter is used for accept list in which user define objects get stored
Comparator: this parameter decide runtime object of Comparator implementer class
Example with source code

```
package org.techhub;
import java.util.*;
public class ComparatorApplication {
        public static void main(String[] args) {
                List<Employee> list= new ArrayList<Employee>();
                list.add(new Employee("A",3,3000));
                list.add(new Employee("B",4,1000));
                list.add(new Employee("C",2,2000));
                list.add(new Employee("D",1,5000));
                list.add(new Employee("E",4,4000));
                System.out.println("Show Employee Data");
                for(Employee e:list) {
                        System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
                }
                Comparator <Employee>c=new SortById();

                Collections.sort(list,c);
                System.out.println("Sort data by using id");
                for(Employee e:list) {
                        System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
                }
                c=new SortBySal();
                Collections.sort(list,c);
                System.out.println("Sort data by using sal");
                for(Employee e:list) {
                        System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
                }
        }
}
```

Set Collection

Set Collection is used for store unique data in collection and not allow duplicated data and there is changes to generate the random data
Set collection internally use hashcode for compare objects if set collection found the same hashcode of object then set not allow data means consider it is duplicated data and if set collection find the duplicated hashcode then set allow data

There are three types of Set implementer classes

---

a. **HashSet :** HashSet collection can store unique data but generate random sequences of data and internally HashSet collection can use the HashMap as backend data structure

   If we want to work with HashSet we have some following constructor

   Constructor of HashSet collection

   ---

   1. **public HashSet():** this constructor is used for create hashset with default capacity 16 and load factor 0.75 and internally use HashMap as data structure

   Q. What is the load factor?
   load factor is threshold value which decide when HashMap or HashSet internally increase its size doubling

   2. **HashSet(Collection):** this constructor is used for copy data from another collection and store in HashSet collection

   Example with source code

```
package org.techhub;
import java.util.*;
public class TestHashSetApplication {
	public static void main(String[] args) {
		ArrayList <Integer> al = new ArrayList<Integer>();
		al.add(10);
		al.add(20);
		al.add(30);
		al.add(40);
		al.add(10);
		al.add(20);
		al.add(30);
		al.add(40);
		HashSet hs= new HashSet(al);
		System.out.println(hs);
	}

}
```

   3. **HashSet(int initialCapacity,float loadFactor)**: this constructor is used for set user defined capacity as well as set user defined load factor

4. **HashSet(int initialCapacity):** this constructor can set initial capacity with default load factor 0.75.

Example of HashSet



b. **LinkedHashSet :**LinkedHashSet collection can store unique data but store data as per user sequence and use LinkedHashMap as backend data structure

Constructor of LinkedHashSet

LinkedHashSet(): Create new empty constructor with a default capacity 16 and load factor 0.75
LinkedHashSet(int initialCapacity): create LinkedHashSet with user defined initial capacity
LinkedHashSet(int initialCapacity, float loadFactor): this constructor is used for create LinkedHashSet with user define initial capacity and user define load factor
LinkedHashSet(Collection): copy data from other collection and store in LinkedHashSet object

Example with source code

```
package org.techhub;
import java.util.*;
public class TestHashSetApplication {
	public static void main(String[] args) {
		LinkedHashSet <Integer>hs = new LinkedHashSet<Integer>();
		hs.add(10);
		hs.add(20);
		hs.add(300);
		hs.add(400);
		hs.add(40);
		hs.add(50);
		hs.add(20);
```

```
        hs.add(200);
        Iterator<Integer> i=hs.iterator();
        while(i.hasNext()) {
            Integer val=i.next();
            System.out.println(val);
        }
    }
}
```

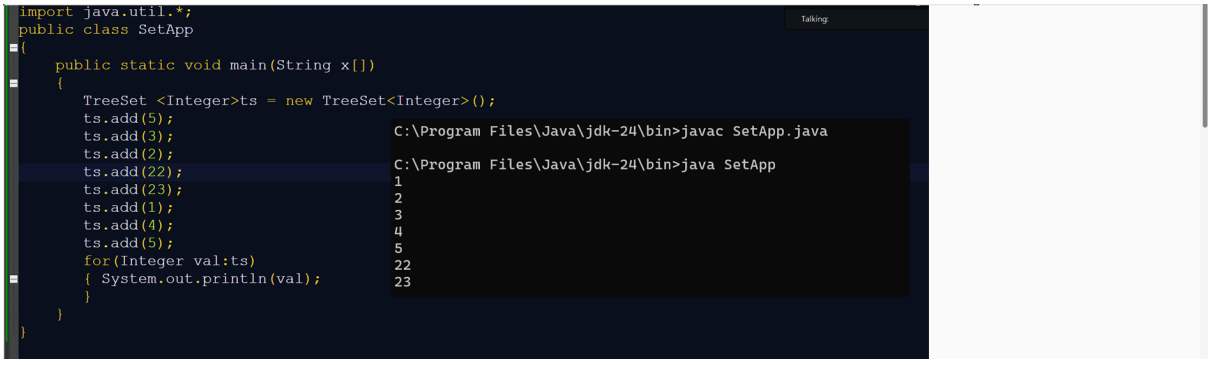
c. **TreeSet :** TreeSet collection and store data in ascending order or arrange data in ascending order

public java.util.TreeSet(): Create empty tree and can sort only primitive type of data

public java.util.TreeSet(java.util.Comparator<? super E>): this constructor help us to sort the user define objects by using Comparator interface

public java.util.TreeSet(java.util.Collection<? extends E>): this constructor is used for copy data from other collection and store TreeSet and sort it

Example with default constructor



Example with TreeSet(Comparator) constructor

```
package org.techhub;

import java.util.*;

class Student {
    private int id;
    private String name;

    public Student() {

    }
```

```
	public Student(String name, int id, int per) {
		this.name = name;
		this.id = id;
		this.per = per;

	}

	public int getId() {
		return id;
	}

	public void setId(int id) {
		this.id = id;
	}

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

	public int getPer() {
		return per;
	}

	public void setPer(int per) {
		this.per = per;
	}

	private int per;
}

class SortStudentById implements Comparator<Student> {

	@Override
	public int compare(Student o1, Student o2) {
		if (o1.getId() > o2.getId()) {
			return 1;
		} else if (o1.getId() < o2.getId()) {
			return -1;
		} else {
			return 0;
		}
	}
```

```
}

public class TestHashSetApplication {
	public static void main(String[] args) {
	Comparator<Student> sid=new SortStudentById();
		TreeSet<Student> ts = new TreeSet<Student>(sid);
		ts.add(new Student("ABC", 1, 80));
		ts.add(new Student("MNO", 3, 60));
		ts.add(new Student("PQR", 2, 90));
		ts.add(new Student("STV", 5, 700));
		ts.add(new Student("XYZ", 4, 98));

		Iterator<Student> i = ts.iterator();
		while (i.hasNext()) {
			Student s = i.next();
	System.out.println(s.getId() + "\t" + s.getName() + "\t" + s.getPer());
		}
	}
}
```

Or

```
package org.techhub;

import java.util.*;

class Student {
	private int id;
	private String name;

	public Student() {

	}

	public Student(String name, int id, int per) {
		this.name = name;
		this.id = id;
		this.per = per;

	}

	public int getId() {
		return id;
	}

	public void setId(int id) {
		this.id = id;
	}
```

```
	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

	public int getPer() {
		return per;
	}

	public void setPer(int per) {
		this.per = per;
	}

	private int per;
}



public class TestHashSetApplication {
	public static void main(String[] args) {
      Comparator<Student> sid=new Comparator<Student>() {
			@Override
			public int compare(Student o1, Student o2) {

				return o1.getId()>o2.getId()?1:o1.getId()<o2.getId()?-1:0;

			}
		};
		TreeSet<Student> ts = new TreeSet<Student>(sid);
		ts.add(new Student("ABC", 1, 80));
		ts.add(new Student("MNO", 3, 60));
		ts.add(new Student("PQR", 2, 90));
		ts.add(new Student("STV", 5, 700));
		ts.add(new Student("XYZ", 4, 98));

		Iterator<Student> i = ts.iterator();
		while (i.hasNext()) {
			Student s = i.next();
			System.out.println(s.getId() + "\t" + s.getName() + "\t" + s.getPer());
		}

	}
}
Or
package org.techhub;
```

```
import java.util.*;

class Student {
	private int id;
	private String name;

	public Student() {

	}

	public Student(String name, int id, int per) {
		this.name = name;
		this.id = id;
		this.per = per;

	}

	public int getId() {
		return id;
	}

	public void setId(int id) {
		this.id = id;
	}

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

	public int getPer() {
		return per;
	}

	public void setPer(int per) {
		this.per = per;
	}

	private int per;
}



public class TestHashSetApplication {
```

```
	public static void main(String[] args) {

		TreeSet<Student> ts = new TreeSet<Student>((Student o1, Student
o2)->o1.getId()>o2.getId()?1:o1.getId()<o2.getId()?-1:0);
		ts.add(new Student("ABC", 1, 80));
		ts.add(new Student("MNO", 3, 60));
		ts.add(new Student("PQR", 2, 90));
		ts.add(new Student("STV", 5, 700));
		ts.add(new Student("XYZ", 4, 98));

		Iterator<Student> i = ts.iterator();
		while (i.hasNext()) {
			Student s = i.next();
			System.out.println(s.getId() + "\t" + s.getName() + "\t" + s.getPer());
		}

	}
}
```

Note: NavigableSet is interface for arrange data in descending order and if we want to get reference of NavigableSet set we have descendingSet() method of TreeSet collection

```
package org.techhub;
import java.util.*;
public class TestHashSetApplication {
	public static void main(String[] args) {

		 TreeSet<Integer> ts= new TreeSet<Integer>();
		 ts.add(5);
		 ts.add(2);
		 ts.add(33);
		 ts.add(23);
		 ts.add(3);
		 ts.add(1);
		NavigableSet<Integer> navSet=ts.descendingSet();
		for(Integer val:navSet) {
			System.out.println(val);
		}
	}
}
```

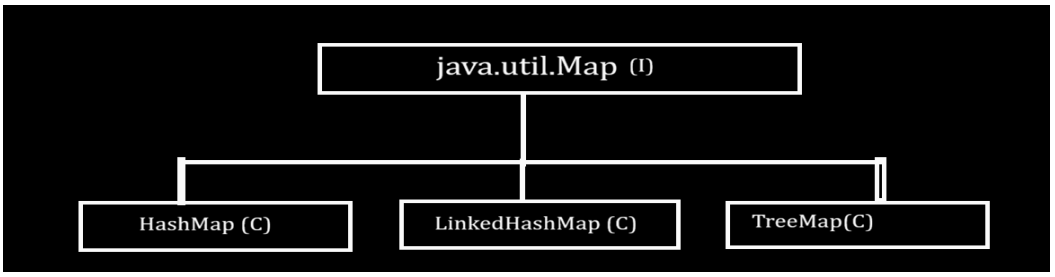
Map interface

Map is not a part of Collection but works as collection and it is used for store data in the form of key and value pair.

Key cannot duplicated and value may be duplicated or key is unique means internally we can say map is combination of set and list collection

Map is recommended when we have duplicated data but we want to maintain the uniqueness in that So internally key work as set and value as List.

If we want to work with Map we have some inbuilt interfaces and some implementer classes



If we want to work with Map interface we have some inbuilt methods provided by Map to us for work with Map

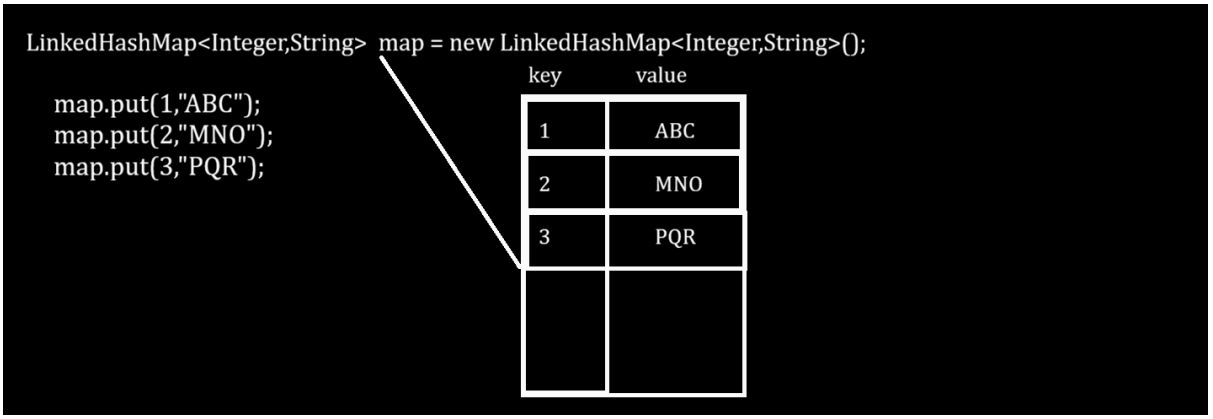
Methods of Map Collection

---

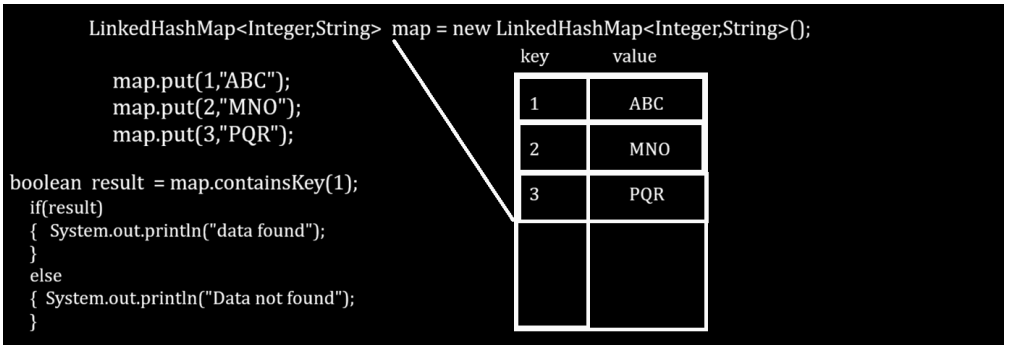
public abstract int size(): this method can return number of element present in map or size of map

public abstract boolean isEmpty(): this method is used for check map is empty or not if empty return true otherwise return false

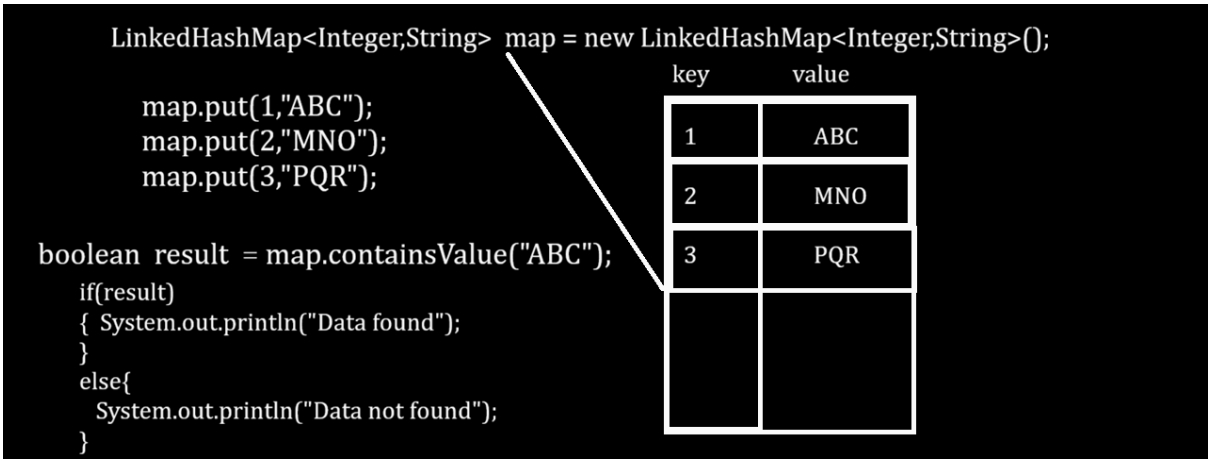
public abstract V put(K, V): this method is used for store data in the form of key and value pair



public abstract boolean containsKey(java.lang.Object): this method searches data in map by using key if key found return true otherwise return false.



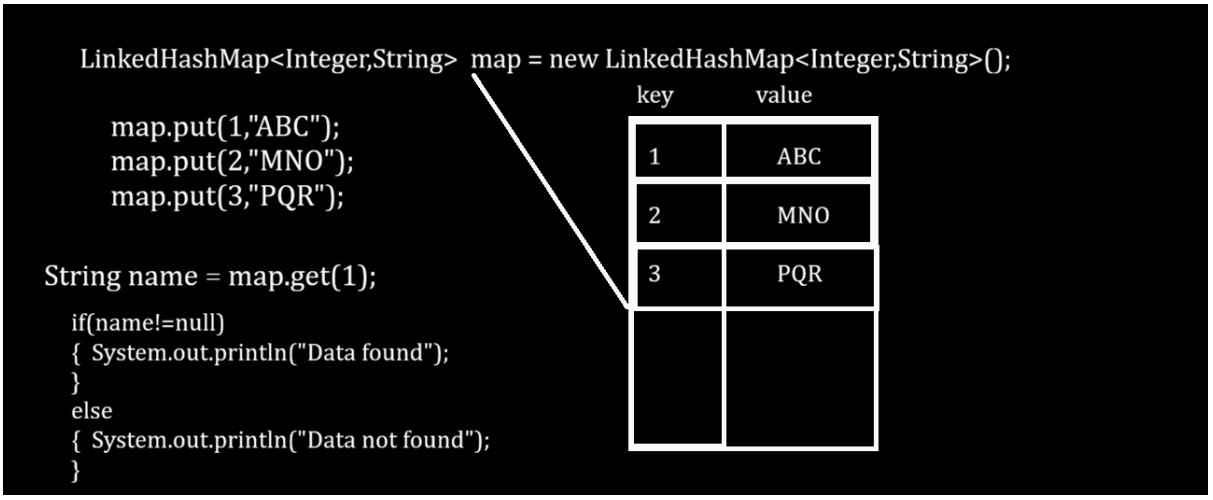
public abstract boolean containsValue(java.lang.Object): this method can search data from map by using its value if value found return true otherwise return false.



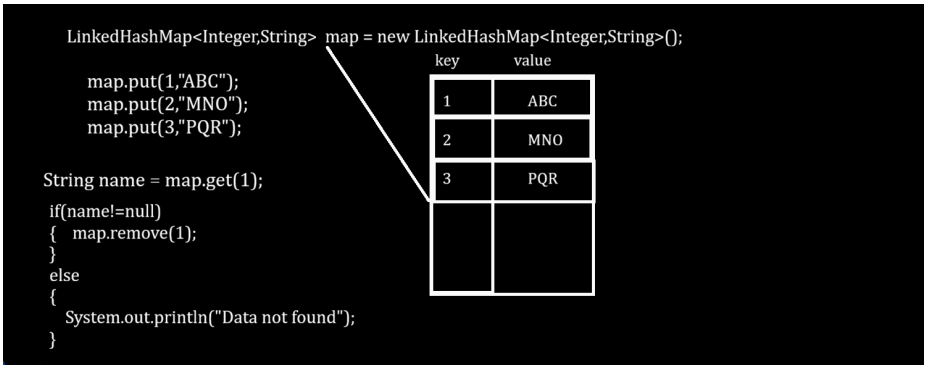
public abstract V get(java.lang.Object): this method is used for accept key and return its value if key found return null value

Note: this method is recommended in two cases

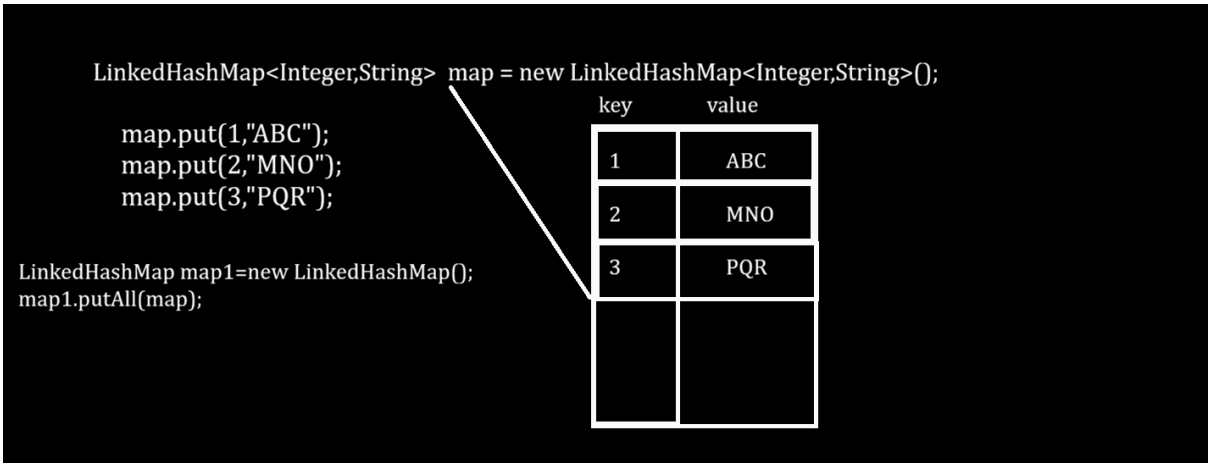
a. For fetching data from map
b. For search data from map.



public abstract V remove(java.lang.Object): this method is used for remove data from map by using its key and return remove data but if key not found then generate Exception at runtime

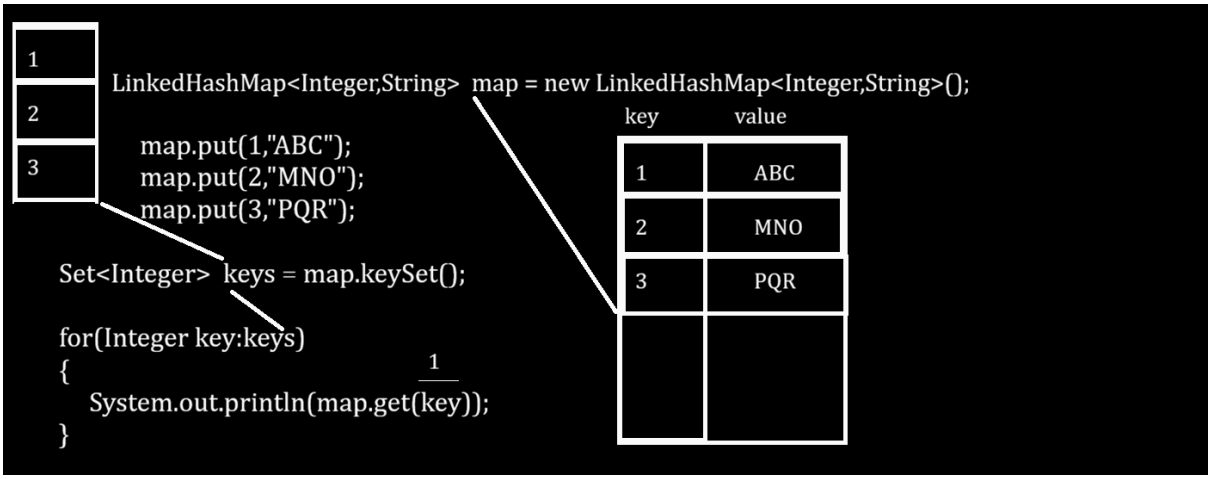


public abstract void putAll(java.util.Map<? extends K, ? extends V>): this is used for copying data from one map to another map.

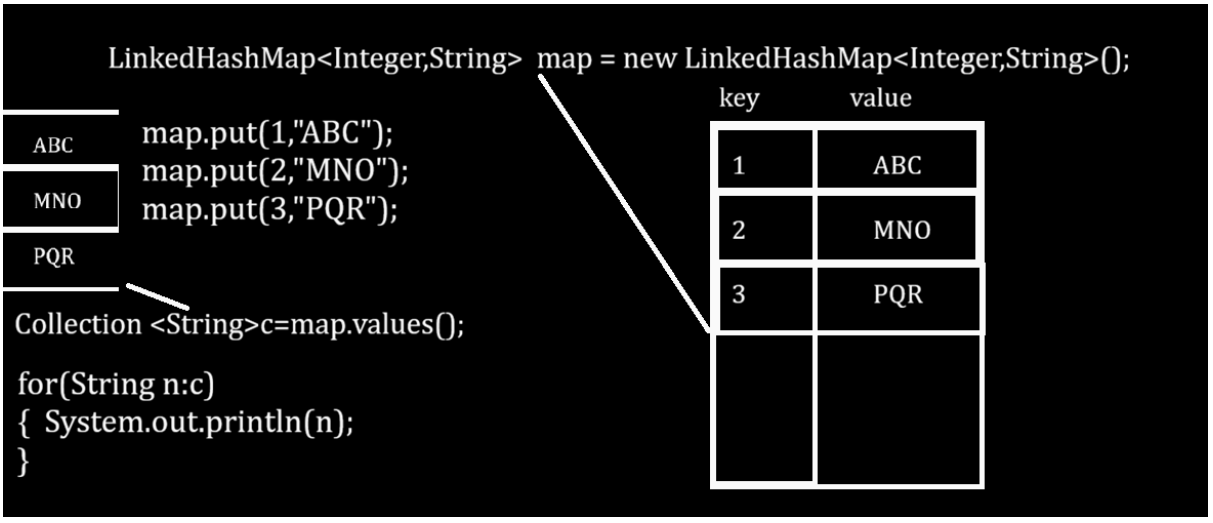


public abstract void clear(): remove the all data of map

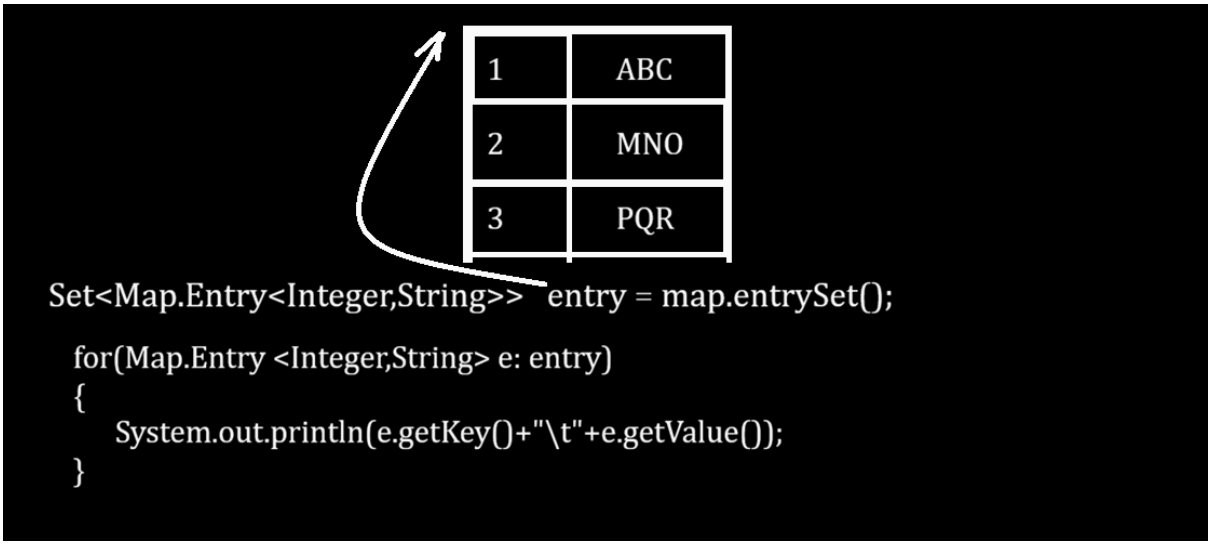
public abstract java.util.Set<K> keySet(): this method is used for fetch all keys and return reference of Set interface



public abstract java.util.Collection<V> values(): values method return all values with Collection interface reference



public abstract java.util.Set<java.util.Map.Entry<K, V>> entrySet(): this method help us to return complete record from map interface means return key as well as value



There are three implementer classes of Map interface

HashMap : HashMap can store data in the form key and value pair and generate key randomly or generate random data

LinkedHashMap : LinkedHashMap can store data in the form of key and value pair and generate data sequence according to user sequence

TreeMap : TreeMap can store data in the form of key and value pair and generate key using ascending order

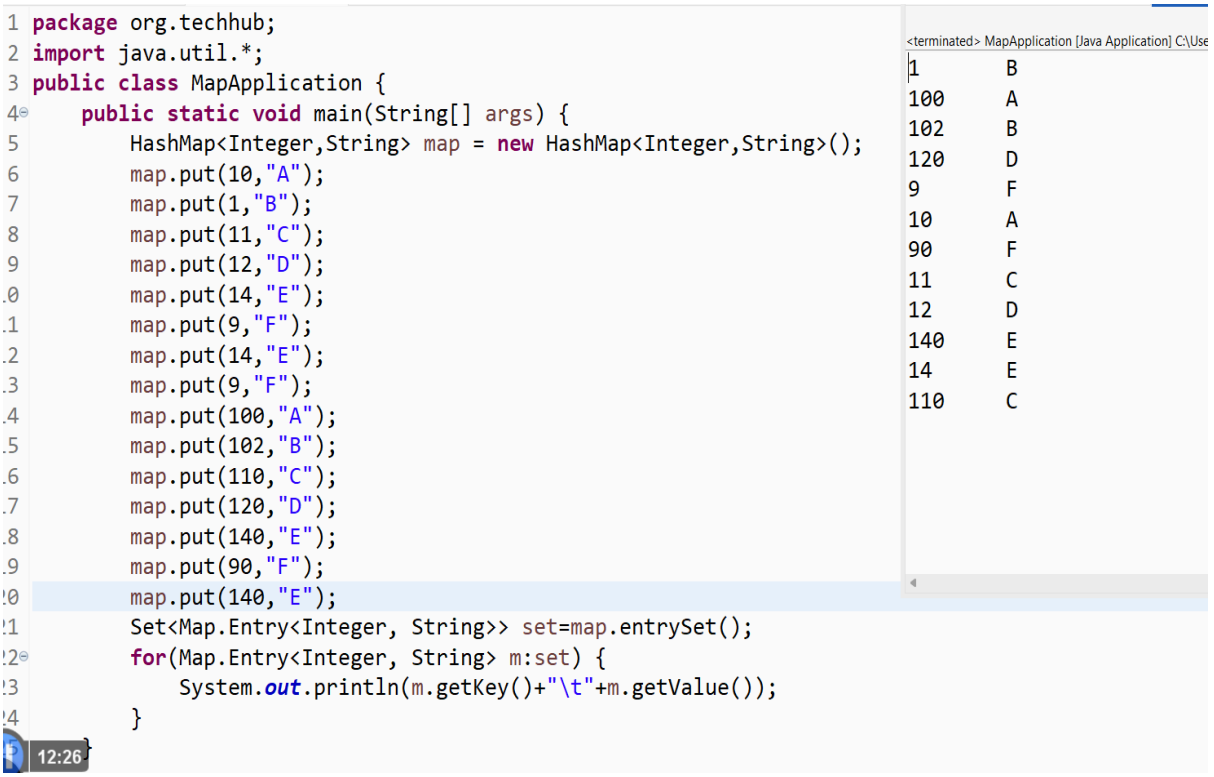
HashMap Collection

Constructors of HashMap

1. **public java.util.HashMap():** this constructor is used for create HashMap with a default capacity 16 and load factor 0.75
   Load factor indicate when hashmap should increase its capacity
   Means according to this constructor when we add 13th element in HashMap then HashMap automatically doubling the capacity

2. **public java.util.HashMap(int):** this constructor is used for create HashMap with initial capacity set by user with a default load factor i.e 0.75
3.
4. **public java.util.HashMap(int, float): this constructor contain two parameter**
   int : this parameter indicate the initial capacity set by user
   float: this parameter indicate the load factor or user define load factor

5. **public java.util.HashMap(java.util.Map<? extends K, ? extends V>):** this constructor is used for copy the data from another map and store in HashMap



```java
package org.techhub;
import java.util.*;
public class MapApplication {
    public static void main(String[] args) {
        HashMap<Integer,String> map = new HashMap<Integer,String>();
        map.put(10,"A");
        map.put(1,"B");
        map.put(11,"C");
        map.put(12,"D");
        map.put(14,"E");
        map.put(9,"F");
        map.put(14,"E");
        map.put(9,"F");
        map.put(100,"A");
        map.put(102,"B");
        map.put(110,"C");
        map.put(120,"D");
        map.put(140,"E");
        map.put(90,"F");
        map.put(140,"E");
        Set<Map.Entry<Integer, String>> set=map.entrySet();
        for(Map.Entry<Integer, String> m:set) {
            System.out.println(m.getKey()+"\t"+m.getValue());
        }
    }
```

```
<terminated> MapApplication [Java Application] C:\Use
1       B
100     A
102     B
120     D
9       F
10      A
90      F
11      C
12      D
140     E
14      E
110     C
```

LinkedHashMap: LinkedHashMap is a child of HashMap but LinkedHashMap can generate the output according to user sequence or insertion order according to user.

Constructor of LinkedHashMap

---

public LinkedHashMap(): create Map with default capacity of 16 and load factor with 0.75

public LinkedHashMap(int initialCapacity): create the LinkedHashMap with initial capacity set by user

public LinkedHashMap(int initialCapacity,float loadFactor): this constructor is used for create LinkedHashMap with initial capacity as per the user capacity and load factor according to user

public LinkedHashMap(int initialCapacity,float loadFactor,boolean flag): this constructor help us set initial capacity as per the user capacity , set load factory according to user and if we set flag as false then just override the value with key not change place of key or insertion order of key but if we set flag as true then change the insertion order of key and set latest key insertion order

LinkedHashMap(Map): this constructor helps us to copy data from other maps and stored LinkedHashMap.

```java
package org.techhub;
import java.util.*;
public class MapApplication {
    public static void main(String[] args) {
        LinkedHashMap<Integer,String> map = new LinkedHashMap<Integer,String>();
        map.put(10,"A");
        map.put(1,"B");
        map.put(11,"C");
        map.put(12,"D");
        map.put(14,"E");
        map.put(9,"F");
        map.put(14,"T");
        map.put(9,"F");
        map.put(100,"A");
        map.put(102,"B");
        map.put(110,"C");
        map.put(120,"D");
        map.put(140,"E");
        map.put(90,"F");
        map.put(140,"M");
        Set<Map.Entry<Integer, String>> set=map.entrySet();
        for(Map.Entry<Integer, String> m:set) {
            System.out.println(m.getKey()+"\t"+m.getValue());
        }
    }
}
```

```
<terminated> MapApplication [Java Application] C:\Users\Hp\.p2\pool\plugins\org
10      A
1       B
11      C
12      D
14      T
9       F
100     A
102     B
110     C
120     D
140     M
90      F
```

TreeMap: TreeMap can arrange key in ascending order format

Constructor of TreeMap

TreeMap(): this constructor is used for creating maps and performing sorting with primitive data types keys.

```java
package org.techhub;
import java.util.*;
public class MapApplication {
    public static void main(String[] args) {
        TreeMap <Integer,String> map = new TreeMap<Integer,String>();
        map.put(10,"A");
        map.put(1,"B");
        map.put(11,"C");
        map.put(12,"D");
        map.put(14,"E");
        map.put(9,"F");
        map.put(14,"T");
        map.put(9,"F");
        map.put(100,"A");
        map.put(102,"B");
        map.put(110,"C");
        map.put(120,"D");
        map.put(140,"E");
        map.put(90,"F");
        map.put(140,"M");
        Set<Map.Entry<Integer, String>> set=map.entrySet();
        for(Map.Entry<Integer, String> m:set) {
            System.out.println(m.getKey()+"\t"+m.getValue());
        }
    }
}
```

```
<terminated> MapApplication [Java Application] C:\Users\Hp\.p2\pool\p
1       B
9       F
10      A
11      C
12      D
14      T
90      F
100     A
102     B
110     C
120     D
140     M
```

TreeMap(Comparator): this constructor is used for sort data when we add key as user define objects

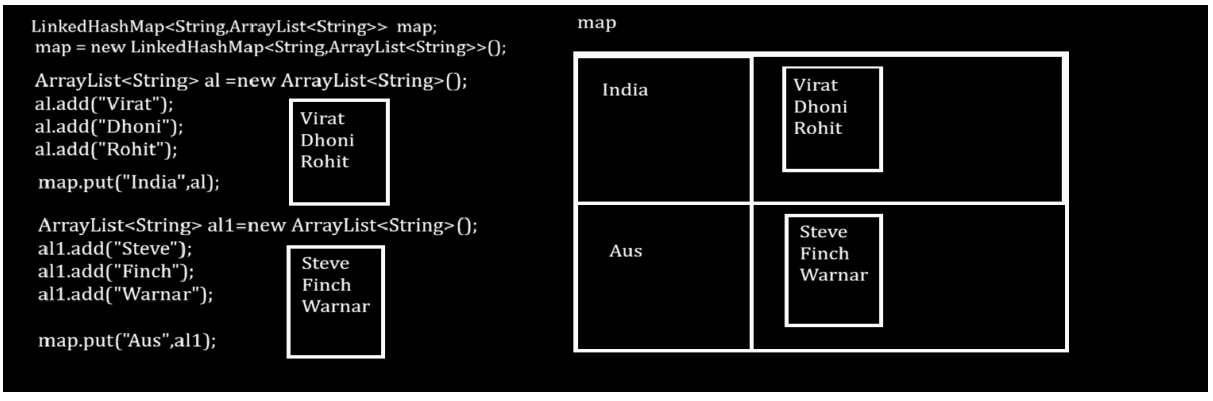
TreeMap(Map): copy data from other map and store in Map and sort it

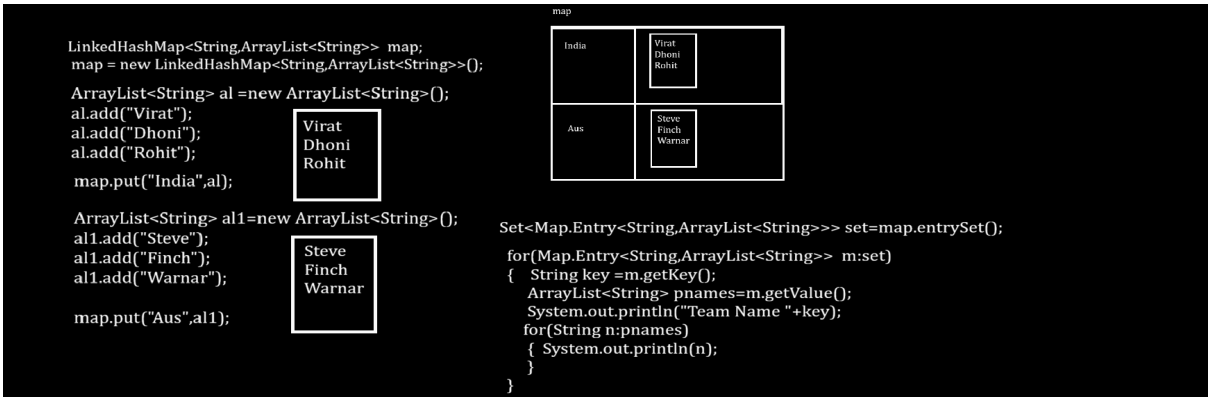
Example:WAP to find the frequency of array data or array elements or occurrence of array data

```
package org.techhub;
import java.util.*;
public class MapApplication {
	public static void main(String[] args) {
		int arr[] = new int[] { 10, 20, 30, 10, 20, 40, 50, 30, 10, 20 };
LinkedHashMap<Integer, Integer> map = new LinkedHashMap<Integer, Integer>();
		for (int val : arr) {
			Integer count = map.get(val);
			if (count == null) {
				count = 0;
			}
			++count;
			map.put(val, count);
		}
		System.out.println("Display the occurence of data");
		Set<Map.Entry<Integer, Integer>> set = map.entrySet();
		for (Map.Entry<Integer, Integer> m : set) {
			System.out.println(m.getKey() + "\t" + m.getValue());
		}
	}

}
```

Example: Map with collection framework





Example with source code

```
package org.techhub;

import java.util.*;

public class MapApplication {
	public static void main(String[] args) {
		LinkedHashMap<String, ArrayList<String>> map = new
LinkedHashMap<String, ArrayList<String>>();
		ArrayList <String> al = new ArrayList<String>();
		al.add("Virat");
		al.add("Rohit");
		al.add("Dhoni");

		map.put("India", al);

		ArrayList <String> al1 = new ArrayList<String>();
		al1.add("Smith");
		al1.add("Warnar");
		al1.add("Finch");
			map.put("AUS", al1);
		Set<Map.Entry<String, ArrayList<String>>> set=map.entrySet();
		for(Map.Entry<String, ArrayList<String>> m:set) {
			System.out.println("Team Name "+m.getKey());
			ArrayList<String> a=m.getValue();
			for(String n:a) {
				System.out.println(n);
			}
		}
	}
}
```

Example of Map within map



```
package org.techhub;
import java.util.*;
public class MapApplication {
	public static void main(String[] args) {
	 LinkedHashMap<String, LinkedHashMap<Integer, String>> dept = new
LinkedHashMap<String, LinkedHashMap<Integer,String>>();

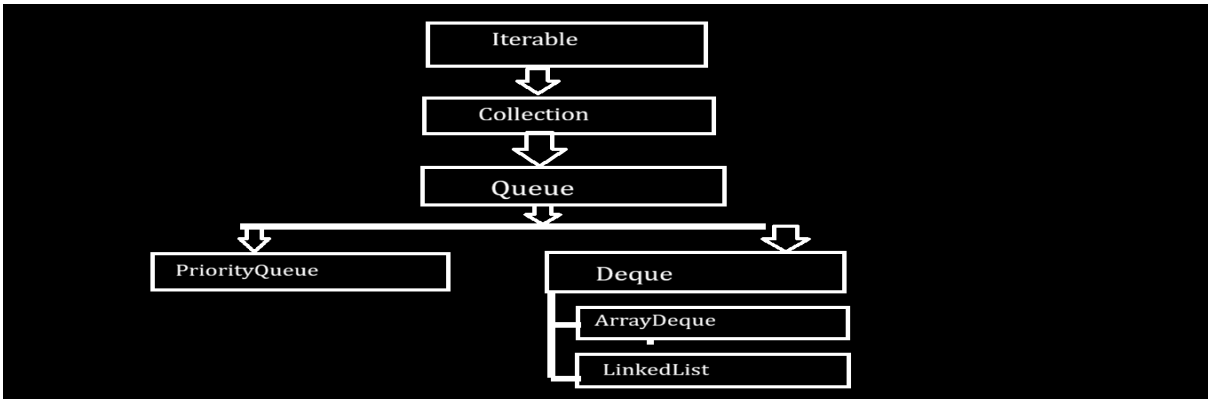
	 LinkedHashMap<Integer,String> FE = new LinkedHashMap<Integer, String>();
	 FE.put(1, "A");
	 FE.put(2, "B");
	 FE.put(3, "C");

	 dept.put("FE", FE);

	 LinkedHashMap<Integer,String> SE = new LinkedHashMap<Integer, String>();
	 SE.put(1, "C");
	 SE.put(2, "D");
	 SE.put(3, "E");
	 dept.put("SE", SE);
	 Set<Map.Entry<String, LinkedHashMap<Integer, String>>> set=dept.entrySet();
	 for(Map.Entry<String, LinkedHashMap<Integer, String>> m:set)
	 { System.out.println(m.getKey());
		System.out.println("=========================");
		LinkedHashMap<Integer, String> c=m.getValue();
		Set<Map.Entry<Integer, String>> sdata=c.entrySet();
		for(Map.Entry<Integer, String> s:sdata) {
			System.out.println(s.getKey()+"\t"+s.getValue());
		}
	 }

	}
}
```

Queue : Queue data structure holds data in first in first out format - the element added first is usually removed first.



You can create Queue like as

a. **Create Queue using a LinkedList**

Example: Queue <Integer> q = new LinkedList<Integer>();
q.add(10);
q.add(20);
q.add(30);

Methods of Queue interface

---

public abstract boolean add(E): this method is used for add element in at end of queue and return true if element added otherwise return throws exception

public abstract boolean offer(E): add an element and return true otherwise return false

public abstract E remove(): remove and return head element i.e first and throws exception if queue is empty

public abstract E poll(): this method also remove head element i.e first element and return null if queue is empty

public abstract E element(): return head without removing it throw exception if empty

public abstract E peek(): return a head without removing it return null if empty

Example of Queue with LinkedList

```
package org.techhub.que;
import java.util.*;
public class QueueApplication {
   public static void main(String[] args) {
      Queue<Integer> q = new LinkedList<Integer>();
                  q.add(10);
                  q.add(20);
                  q.offer(30);
                  System.out.println("Before element remove");
                  Iterator<Integer> i = q.iterator();
                  while (i.hasNext()) {
                        Object obj = i.next();
                        System.out.print(obj + "\t");
                  }
                  System.out.print("\nRemove element " + q.poll());
                  System.out.println("\nAfter element remove");
                  i = q.iterator();
                  while (i.hasNext()) {
                        Object obj = i.next();
                        System.out.print(obj + "\t");
                  }
         }
}
```

ArrayDeque: ArrayDeque is a resizable array implementation of the Deque interface in Java. Deque means Double Ended Queue so you can add and remove elements from both ends

Methods of ArrayDeque

SRNO	Method name	Description
1	addFirst(E):	Add at beginning and if fail return exception
2	addLast(E)	Add at end if fail return exception
3	offerFirst(E)	Add at beginning and if fail return false
4	offerLast(E)	Add at end and if fail return false
5	removeFirst()	Remove at beginning and return exception if empty
6	removeLast()	Remove at end and return exception if empty
7	poolFirst():	Remove at beginning and return null if empty
8	poolLast)	Remove at end and return null if empty
9	peekFirst()	View first element
10	peekLast)	View last element

Example of ArrayDeque

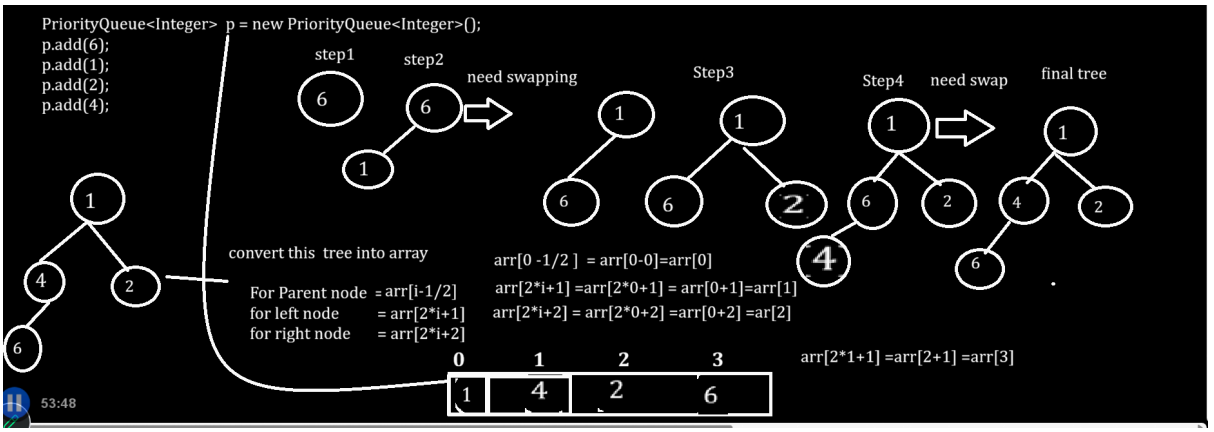
```
package org.techhub.que;
import java.util.*;
public class QueueApplication {
  public static void main(String[] args) {
      ArrayDeque<Integer> q = new ArrayDeque<Integer>();
       q.addFirst(10);
       q.addLast(20);
       q.addFirst(30);
       System.out.println(q);
         }
}
```

PriorityQueue :

If we think about PriorityQueue of java it is developed by using min heap concept
Min is a concept in tree structure where parents should be min value.

Important points related with min value

a. It has to be complete binary tree and entire tree has two children and all children should start with its left self
b. Parent node should always be the min value of two children




```java
package org.techhub.que;
import java.util.*;
public class QueueApplication {
   public static void main(String[] args) {

       PriorityQueue<Integer> p = new PriorityQueue<Integer>();
       p.add(6);
       p.add(1);
       p.add(2);
       p.add(4);

       System.out.println("display data");
       for(Integer val:p) {
           System.out.print(val+"\t");
       }


    }
}

```

```
<terminated> QueueApplication [Java Application] C:\Users\Hp\.p2\pool\plugins\o
display data
1       4       2       6
```

Internal working of HashMap

Suppose consider if we create HashMap like as HashMap<String,String> map=new HashMap<String,String>() the JVM creates 16 bucket slots for hashmap objects and you can store more than 16 values in hashmap the concept is load factor.
Means when HashMap reaches more than 75% or 0.75 of its capacity then double the existing capacity
Means when we add13th element in HashMap then HashMap can increase its capacity from 16 to 32

Now we want to see what is bucket and how data store in bucket

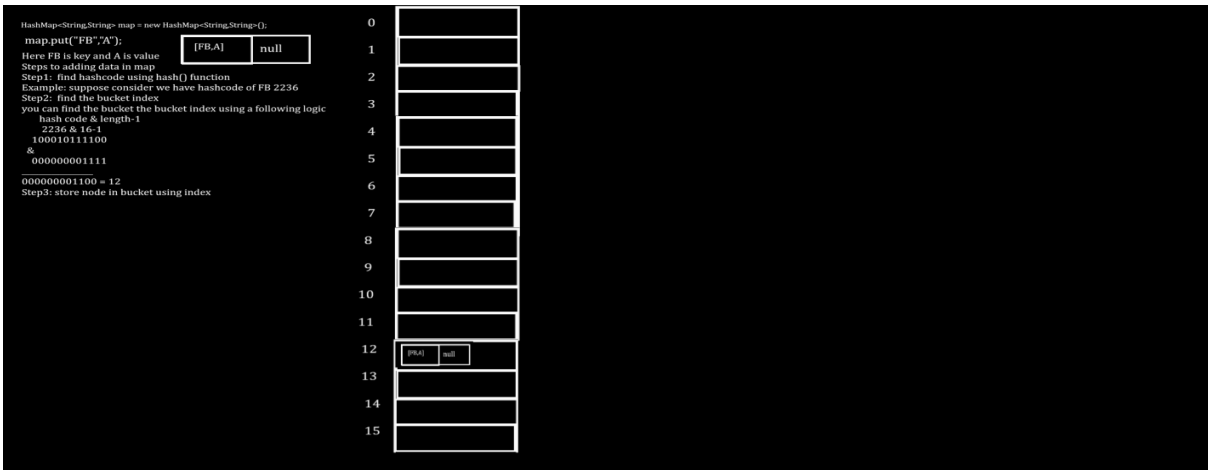
---

Bucket is internally arrayed and can store data in the form of LinkedList and LinkedList contain data in the form of node and address.

Lets see what happen when add node in linked list
Initially there is no node in LinkedList support first time we want to add first node with item-1 then your look look like as



Now suppose consider we are going to store data in map by using put() method like as



Now we want to one more element in Map



Now we want to discuss about the concept of hash collision

Q. What is hash collision?

Collision occur when two different keys produce the same hashcode then we need to store keys in same bucket or on same index in bucket then JVM compare newly generated key same as previous generated key or compare new node key with previous node key which is present on same buck if new node key match with previous key then override new with previous node on same index otherwise add new node on same index after previous node and store next node address in previous node next address called as hash collision

Example;



How get() method works internally

---

get() method of HashMap return value using a key fast
Example: map.get("LD") so the time complexity of get() is O(1)

Steps

1. **JVM find the hashcode of key**
2. **Find the bucked index using hashcode**
3. **Go to the bucket index and return data**

Note: HashMap internally store only one node in bucket if if there is hash collision then there possibility store more than one node in single bucket of HashMap so there is possible we need to iterate linked list for searching data in bucket there may be possibility to degrade performance
So java 8 enhance this HashMap by using red black tree concept

Object class